Java8新特性

1.Lambda表达式

1.1Lambda表达式简介

- Lambda表达式是一个匿名函数，可以理解为一段可以传递的代码。
- Lambda表达式的优点：简化了匿名内部类的使用，语法更加简单。
- 匿名内部类语法冗余，体验了Lambda表达式后，发现Lambda表达式是简化匿名内部类的一种方式。

1.2Lambda的语法规则

- Lambda省去了面向对象的条条框框，Lambda的标准格式由3个部分组成

```
(参数类型 参数名称) -> {
        代码体;
}
```

格式说明：

- (参数类型 参数名称):参数列表
- {代码体;} :方法体
- -> : 箭头，分割参数列表和方法体

1.3 @FunctionalInterface注解

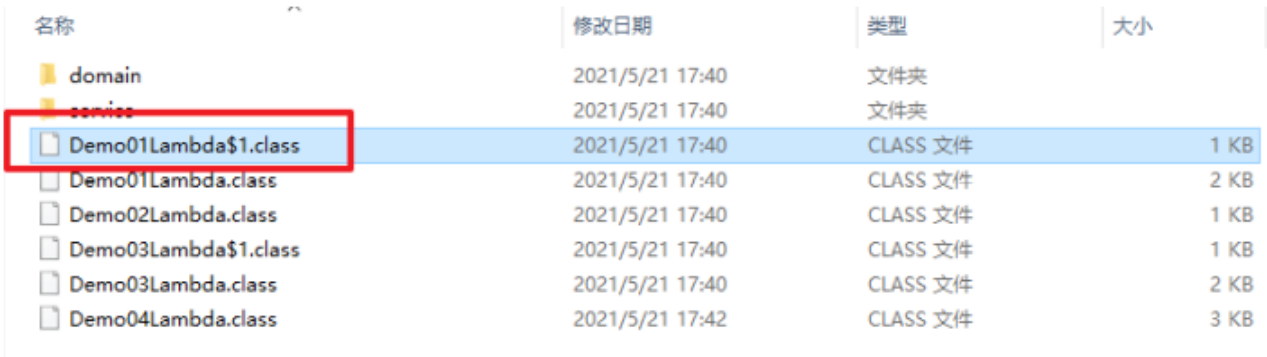
```
/**
* @FunctionalInterface
* 这是一个标志注解，被该注解修饰的接口只能声明一个抽象方法
*/
@FunctionalInterface
public interface UserService {
    void show();
}
```

1.4 Lambda表达式的原理

- 匿名内部类原理

  匿名内部类的本质是在编译时生成一个Class 文件。XXXXX$1.class

```
public class Demo01Lambda {
   public static void main(String[] args) {
   // 开启一个新的线程
   new Thread(new Runnable() {
       @Override
       public void run() {
          System.out.println("新线程中执行的代码 :
          "+Thread.currentThread().getName());
       }
    }).start();
   System.out.println("主线程中的代码: " + Thread.currentThread().getName());
   System.out.println("---------------");

   /*new Thread(() -> { System.out.println("新线程Lambda表达
式..."+Thread.currentThread().getName()); }).start();*/
  }
}
```



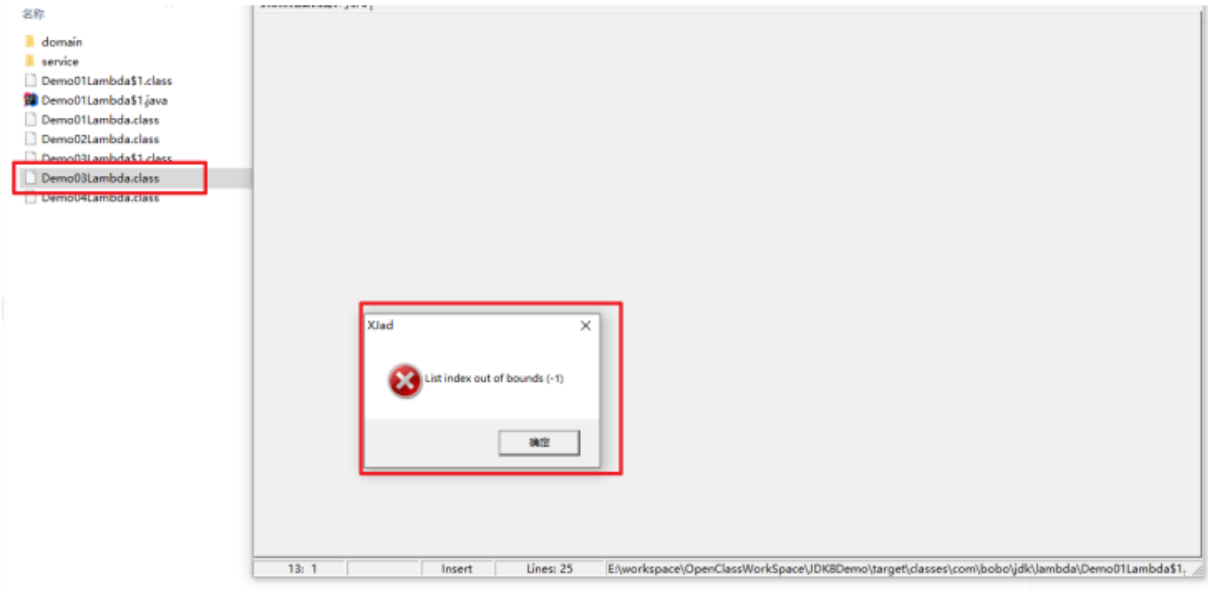
名称	修改日期	类型	大小
domain	2021/5/21 17:40	文件夹	
service	2021/5/21 17:40	文件夹	
Demo01Lambda$1.class	2021/5/21 17:40	CLASS 文件	1 KB
Demo01Lambda.class	2021/5/21 17:40	CLASS 文件	2 KB
Demo02Lambda.class	2021/5/21 17:40	CLASS 文件	1 KB
Demo03Lambda$1.class	2021/5/21 17:40	CLASS 文件	1 KB
Demo03Lambda.class	2021/5/21 17:40	CLASS 文件	2 KB
Demo04Lambda.class	2021/5/21 17:42	CLASS 文件	3 KB

还可以通过反编译工具来查看生成的代码 XJad 工具来查看

```
static class Demo01Lambda$1 implements Runnable {
   public void run()
   {
     System.out.println((new StringBuilder()).append("新线程中执行的代码 : "
     ).append(Thread.currentThread().getName()).toString());
   }
    Demo01Lambda$1()
    {
    }
}
```

- Lambda表达式的原理

  通过反编译工具来查看



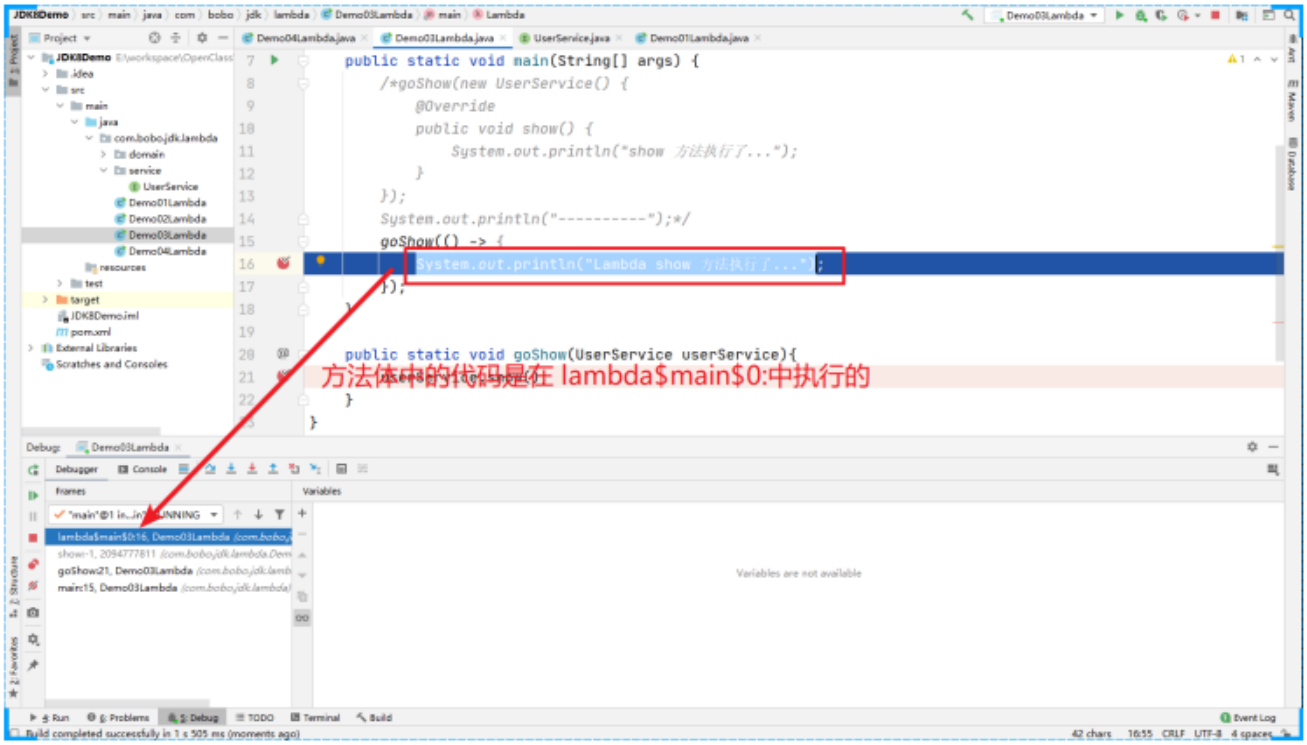
写的有Lambda表达式的class文件，我们通过XJad查看报错。这时我们可以通过JDK自带的一个工具：javap 对字节码进行反汇编操作。

```
javap -c -p 文件名.class
-c:表示对代码进行反汇编
-p:显示所有的类和成员
```

反汇编的结果：

```
E:\workspace\OpenClassWorkSpace\JDK8Demo\target\classes\com\bobo\jdk\lambda>javap
-c -p Demo03Lambda.class
Compiled from "Demo03Lambda.java"
public class com.bobo.jdk.lambda.Demo03Lambda {
  public com.bobo.jdk.lambda.Demo03Lambda();
   Code:
     0: aload_0
     1: invokespecial #1 // Method java/lang/Object."<init>":
()V
     4: return
 public static void main(java.lang.String[]);
   Code:
   0: invokedynamic #2, 0 // InvokeDynamic #0:show:
   ()Lcom/bobo/jdk/lambda/service/UserService;
   5: invokestatic #3 // Method goShow:
   (Lcom/bobo/jdk/lambda/service/UserService;)V
   8: return
  public static void goShow(com.bobo.jdk.lambda.service.UserService);
   Code:
    0: aload_0
    1: invokeinterface #4, 1 // InterfaceMethod
    com/bobo/jdk/lambda/service/UserService.show:()V
    6: return
  private static void lambda$main$0();
   Code:
    0: getstatic #5 // Field
java/lang/System.out:Ljava/io/PrintStream;
    3: ldc #6 // String Lambda show 方法执行了...
    5: invokevirtual #7 // Method
java/io/PrintStream.println:(Ljava/lang/String;)V
    8: return
}
```

在这个反编译的源码中我们看到了一个静态方法 lambda$main$0()，这个方法里面做了什么事情呢？我们通过debug的方式来查看下：



上面的效果可以理解为如下：

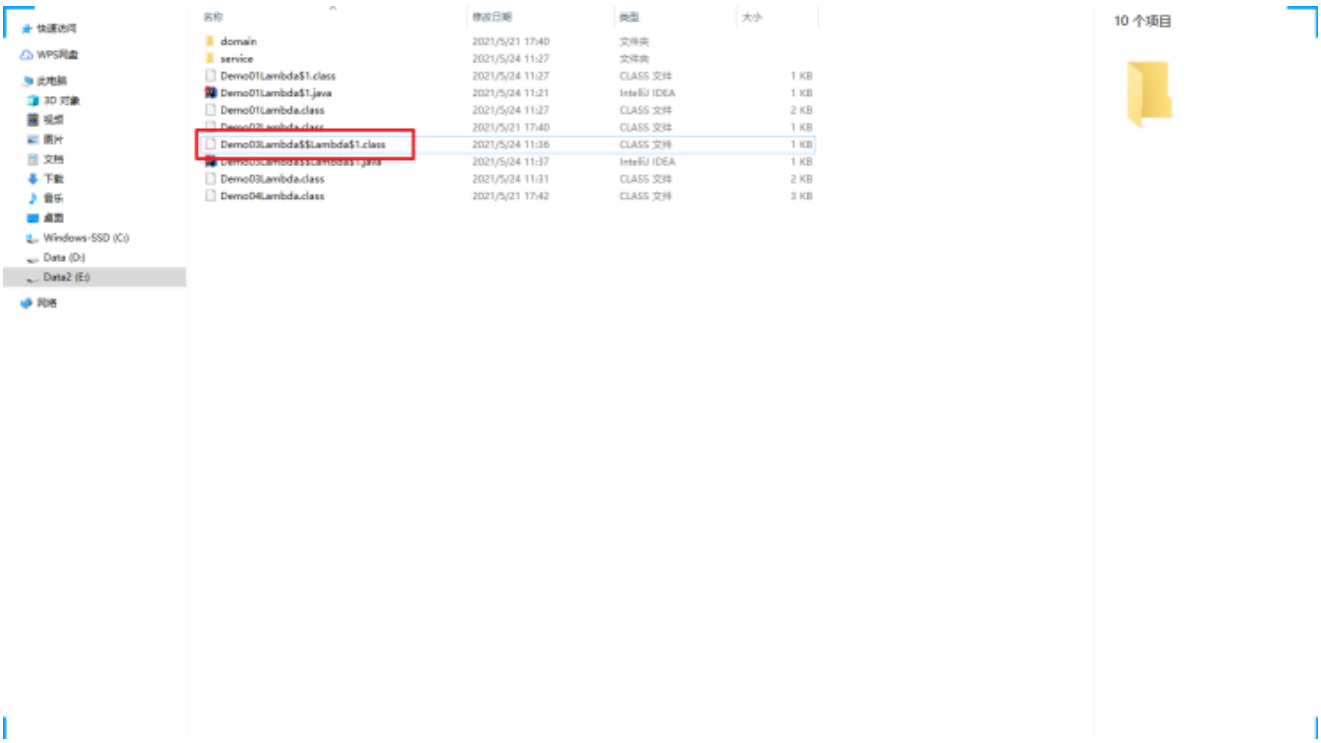
```
public class Demo03Lambda {
   public static void main(String[] args) {
     ....
   }

   private static void lambda$main$0();
     System.out.println("Lambda show 方法执行了...");
   }
}
```

为了更加直观的理解这个内容，我们可以在运行的时候添加 -Djdk.internal.lambda.dumpProxyClasses, 加上这个参数会将内部class码输出到一个文件中

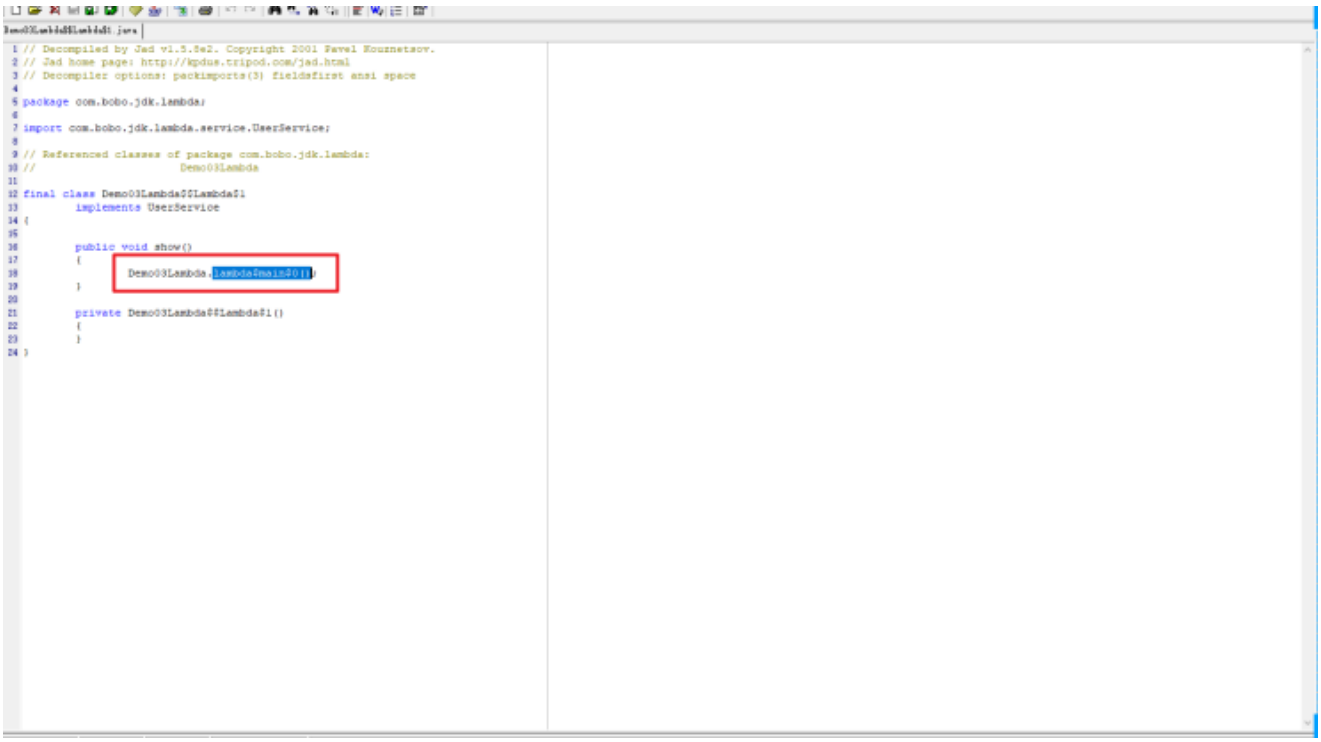
```
java -Djdk.internal.lambda.dumpProxyClasses 要运行的包名.类名
```

命令执行

```
E:\workspace\OpenClassWorkSpace\JDK8Demo\target\classes>java -
Djdk.internal.lambda.dumpProxyClasses com.bobo.jdk.lambda.Demo03Lambda
Lambda show 方法执行了...
```



反编译后的内容：



可以看到这个匿名的内部类实现了UserService接口，并重写了show()方法。在show方法中调用了Demo03Lambda.lambda$main$0(),也就是调用了Lambda中的内容。

```java
public class Demo03Lambda {
   public static void main(String[] args) {
         goShow(new UserService() {
          @Override
          public void show() {
            Demo03Lambda.lambda$main$0();
          }
        });
        System.out.println("----------");
   }

   public static void goShow(UserService userService){
      userService.show();
   }

   private static void lambda$main$0();
      System.out.println("Lambda show 方法执行了...");
   }
}
```

- 小结：

  匿名内部类在编译的时候会产生一个class文件。

  Lambda表达式在程序运行的时候会形成一个类。

1. 在类中新增了一个方法，这个方法的方法体就是Lambda表达式中的代码
2. 还会形成一个匿名内部类，实现接口，重写抽象方法
3. 在接口中重写方法会调用新生成的方法

1.5Lambda表达式省略写法

- 在lambda表达式的标准写法基础上，可以使用省略写法的规则为：

1. 小括号内的参数类型相同可以省略
2. 如果小括号内有且仅有一个参数，则小括号可以省略
3. 如果大括号内有且仅有一个语句，可以同时省略大括号，return 关键字及语句分号。

```java
public class Demo05Lambda {
   public static void main(String[] args) {
         goStudent((String name,Integer age)->{
         return name+age+" 6666 ...";
   });
  // 省略写法
    goStudent((name,age)-> name+age+" 6666 ...");
    System.out.println("------");

    goOrder((String name)->{
       System.out.println("--->" + name);
       return 666;
    });

  // 省略写法
    goOrder(name -> {
      System.out.println("--->" + name);
      return 666;
    });

   goOrder(name -> 666);
}

   public static void goStudent(StudentService studentService){
        studentService.show("张三",22);
   }

   public static void goOrder(OrderService orderService){
      orderService.show("李四");
   }
}
```

1.6 Lambda表达式与匿名内部类对比

- Lambda表达式使用前提

1. 方法的参数或局部变量类型必须为接口才能使用Lambda
2. 接口中有且仅有一个抽象方法(@FunctionalInterface)

- Lambda表达式与匿名内部类对比

1. 所需类型不一样

   匿名内部类的类型可以是 类，抽象类，接口

   Lambda表达式需要的类型必须是接口
2. 抽象方法的数量不一样

   匿名内部类所需的接口中的抽象方法的数量是随意的

Lambda表达式所需的接口中只能有一个抽象方法
3. 实现原理不一样

   匿名内部类是在编译后形成一个class

   Lambda表达式是在程序运行的时候动态生成class

2.接口增强

2.1 JDK1.8中接口的新增

- JDK1.8之前

```
interface 接口名{
    静态常量;
    抽象方法;
}
```

- JDK1.8之后

```
interface 接口名{
    静态常量;
    抽象方法;
    默认方法;
    静态方法;
}
```

2.2 默认方法

2.2.1 为什么要加默认方法

在JDK8以前接口中只能有抽象方法和静态常量，会存在以下的问题：
如果接口中新增抽象方法，那么实现类都必须要重写这个抽象方法，非常不利于接口的扩展的

2.2.2 默认方法语法规则

```
interface 接口名{
   修饰符 default 返回值类型 方法名{
        方法体;
   }
}
```

2.2.3 接口中默认方法的使用

- 接口中的默认方法有两种使用方式

1. 实现类直接调用接口的默认方法
2. 实现类重写接口的默认方法

2.3 静态方法

2.3.1 为什么要加静态方法

- JDK8中为接口新增了静态方法，作用也是为了接口的扩展

2.3.2 静态方法语法规则

```
interface 接口名{
    修饰符 static 返回值类型 方法名{
        方法体;
    }
}
```

2.3.3 接口中静态方法的使用

- 接口中的静态方法在实现类中是不能被重写的，调用的话只能通过接口类型来实现: 接口名.静态方法名();

2.4 默认方法和静态方法区别

1. 默认方法通过实例调用，静态方法通过接口名调用
2. 默认方法可以被继承，实现类可以直接调用接口默认方法，也可以重写接口默认方法
3. 静态方法不能被继承，实现类不能重写接口的静态方法，只能使用接口名调用

3.函数式接口

3.1 函数式接口介绍

- 我们知道使用Lambda表达式的前提是需要有函数式接口，而Lambda表达式使用时不关心接口名，抽象方法名。只关心抽象方法的参数列表和返回值类型。因此为了让我们使用Lambda表达式更加的方法，在JDK中提供了大量常用的函数式接口
- 在JDK中帮我们提供的有函数式接口，主要是在 java.util.function 包中。

3.2 JDK中常用的函数式接口

3.2.1 Supplier

- 无参有返回值的接口，对于的Lambda表达式需要提供一个返回数据的类型。

```
@FunctionalInterface
public interface Supplier<T> {
        /**
         * Gets a result.
         *
         * @return a result
         */
        T get();
}
```

- 代码示例

```java
/**
* Supplier 函数式接口的使用
*/
public class SupplierTest {
    public static void main(String[] args) {
        fun1(()->{
            int arr[] = {22,33,55,66,44,99,10};
            // 计算出数组中的最大值
            Arrays.sort(arr);
            return arr[arr.length-1];
        });
}
    private static void fun1(Supplier<Integer> supplier){
        // get() 是一个无参的有返回值的 抽象方法
        Integer max = supplier.get();
        System.out.println("max = " + max);
    }
}
```

3.2.2 Consumer

- 有参无返回值得接口,前面介绍的Supplier接口是用来生产数据的，而Consumer接口是用来消费数据的，使用的时候需要指定一个泛型来定义参数类型

```java
@FunctionalInterface
public interface Consumer<T> {
    /**
    * Performs this operation on the given argument.
    *
    * @param t the input argument
    */
    void accept(T t);
}
```

- 代码示例

```java
public class ConsumerTest {
    public static void main(String[] args) {
        test(msg -> {
            System.out.println(msg + "-> 转换为小写: " + msg.toLowerCase());
            });
    }
    public static void test(Consumer<String> consumer){
        consumer.accept("Hello World");
    }
}
```

- 接口中默认方法：andThen
    - 如果一个方法的参数和返回值全部是Consumer类型，那么就可以实现效果，消费一个数据的时候，首先做一个操作，然后再做一个操作，实现组合，而这个方法就是Consumer接口中的default方法andThen方法

```
default Consumer<T> andThen(Consumer<? super T> after) {
    Objects.requireNonNull(after);
    return (T t) -> { accept(t); after.accept(t); };
  }
```

- 代码示例

```
public class ConsumerAndThenTest {
    public static void main(String[] args) {
        test2(msg1->{
          System.out.println(msg1 + "-> 转换为小写: " + msg1.toLowerCas())}
        ,msg2->{
          System.out.println(msg2 + "-> 转换为大写: " + msg2.toUpperCase());
      });
  }

  public static void test2(Consumer<String> c1,Consumer<String> c2){
      String str = "Hello World";
      //c1.accept(str); // 转小写
      //c2.accept(str); // 转大写
      //c1.andThen(c2).accept(str);
      c2.andThen(c1).accept(str);
  }
}
```

3.2.3 Function

- 有参有返回值的接口，Function接口是根据一个类型的数据得到另一个类型的数据，前者称为前置条件，后者称为后置条件。有参数有返回值。

```
@FunctionalInterface
public interface Function<T, R> {
    /**
    * Applies this function to the given argument.
    *
    * @param t the function argument
    * @return the function result
    */
    R apply(T t);
}
```

- 代码示例

```java
public class FunctionTest {
    public static void main(String[] args) {
        test(msg ->{
            return Integer.parseInt(msg);
        });
    }

    public static void test(Function<String,Integer> function){
        Integer apply = function.apply("666");
        System.out.println("apply = " + apply);
    }
}
```

- 接口中默认方法：andThen，也是用来进行组合操作

```java
default <V> Function<T, V> andThen(Function<? super R, ? extends V> after) {
    Objects.requireNonNull(after);
    return (T t) -> after.apply(apply(t));
}
```

- 代码示例

```java
public class FunctionAndThenTest {
    public static void main(String[] args) {
        test(msg ->{
            return Integer.parseInt(msg);
        },msg2->{
            return msg2 * 10;
        });
    }

    public static void test(Function<String,Integer>f1,Function<Integer,Integer> f2){
        /*Integer i1 = f1.apply("666");
        Integer i2 = f2.apply(i1);*/
        Integer i2 = f1.andThen(f2).apply("666");
        System.out.println("i2:" + i2);
    }
}
```

- 默认方法compose方法的作用顺序和andThen方法刚好相反
- 静态方法identity，输入什么参数就返回什么参数

3.2.4 Predicate

- 有参且返回值为Boolean的接口

```java
@FunctionalInterface
public interface Predicate<T> {
    /**
    * Evaluates this predicate on the given argument.
    *
    * @param t the input argument
    * @return {@code true} if the input argument matches the predicate,
    * otherwise {@code false}
    */
    boolean test(T t);
}
```

- 代码示例

```java
public class PredicateTest {
    public static void main(String[] args) {
        test(msg -> {
            return msg.length() > 3;
        },"HelloWorld");
    }

    private static void test(Predicate<String> predicate,String msg){
        boolean b = predicate.test(msg);
        System.out.println("b:" + b);
    }
}
```

- 接口中默认方法提供了逻辑关系操作 and or negate isEquals方法

```java
package com.bobo.jdk.fun;
import java.util.function.Predicate;
public class PredicateDefaultTest {
    public static void main(String[] args) {
        test(msg1 -> {
            return msg1.contains("H");
            },msg2 -> {
            return msg2.contains("W");
        });
}
    private static void test(Predicate<String> p1,Predicate<String> p2){
        /*boolean b1 = predicate.test(msg);
        boolean b2 = predicate.test("Hello");*/
        // b1 包含H b2 包含W
        // p1 包含H 同时 p2 包含W
        boolean bb1 = p1.and(p2).test("Hello");
        // p1 包含H 或者 p2 包含W
        boolean bb2 = p1.or(p2).test("Hello");
        // p1 不包含H
        boolean bb3 = p1.negate().test("Hello");
        System.out.println(bb1); // FALSE
        System.out.println(bb2); // TRUE
        System.out.println(bb3); // FALSE
    }
}
```

4.方法引用

4.1 方法引用简介

- 为什么要使用方法引用

  lambda表达式冗余：在使用Lambda表达式的时候，也会出现代码冗余的情况，因为在Lambda表达式中要执行的代码和我们另一个方法中的代码是一样的，这时就没有必要重写一份逻辑了，这时我们就可以“引用”重复代码

4.2 方法引用格式

- 符号表示：::
- 符号说明：双冒号为方法引用运算符，而它所在的表达式被称为方法引用
- 应用场景：如果Lambda表达式所要实现的方案，已经有其他方法存在相同的方案，那么则可以使用方法引用。
- 常见的引用方式：

1. instanceName::methodName 对象::方法名
2. ClassName::staticMethodName 类名::静态方法
3. ClassName::methodName 类名::普通方法
4. ClassName::new 类名::new 调用的构造器
5. TypeName[]::new String[]::new 调用数组的构造器

4.2.1 对象名::方法名

- 这是最常见的一种用法。如果一个类中的已经存在了一个成员方法，则可以通过对象名引用成员方法
- 引用示例

```
public static void main(String[] args) {
    Date now = new Date();
    Supplier<Long> supplier = ()->{return now.getTime();};
    System.out.println(supplier.get());
    // 然后我们通过 方法引用 的方式来处理
    Supplier<Long> supplier1 = now::getTime;
    System.out.println(supplier1.get());
}
```

- 方法引用的注意事项：

1. 被引用的方法，参数要和接口中的抽象方法的参数一样
2. 当接口抽象方法有返回值时，被引用的方法也必须有返回值

4.2.2 类名::静态方法名

- 引用示例

```
public class FunctionRefTest04 {
    public static void main(String[] args) {
        Supplier<Long> supplier1 = ()->{
            return System.currentTimeMillis();
        };
        System.out.println(supplier1.get());
        // 通过 方法引用 来实现
        Supplier<Long> supplier2 = System::currentTimeMillis;
        System.out.println(supplier2.get());
    }
}
```

4.2.3 类名::引用实例方法

- Java面向对象中，类名只能调用静态方法，类名引用实例方法是用前提的，实际上是拿第一个参数作为方法的调用者
- 引用示例

```java
package com.bobo.jdk.funref;
import java.util.Date;
import java.util.function.BiFunction;
import java.util.function.Function;
import java.util.function.Supplier;
public class FunctionRefTest05 {
    public static void main(String[] args) {
        Function<String,Integer> function = (s)->{
            return s.length();
        };
        System.out.println(function.apply("hello"));
        // 通过方法引用来实现
        Function<String,Integer> function1 = String::length;
        System.out.println(function1.apply("hahahaha"));
        BiFunction<String,Integer,String> function2 = String::substring;
        String msg = function2.apply("HelloWorld", 3);
        System.out.println(msg);
    }
}
```

4.2.4 类名::构造器

- 由于构造器的名称和类名完全一致，所以构造器引用使用 ::new 的格式使用

```java
public class FunctionRefTest06 {
    public static void main(String[] args) {
        Supplier<Person> sup = ()->{return new Person();};
        System.out.println(sup.get());
        // 然后通过 方法引用来实现
        Supplier<Person> sup1 = Person::new;
        System.out.println(sup1.get());
        BiFunction<String,Integer,Person> function = Person::new;
        System.out.println(function.apply("张三",22));
    }
}
```

4.2.5 数组::构造器

```java
public static void main(String[] args) {
    Function<Integer,String[]> fun1 = (len)->{
        return new String[len];
    };
    String[] a1 = fun1.apply(3);
    System.out.println("数组的长度是: " + a1.length);
    // 方法引用 的方式来调用数组的构造器
    Function<Integer,String[]> fun2 = String[]::new;
    String[] a2 = fun2.apply(5);
    System.out.println("数组的长度是: " + a2.length);
}
```

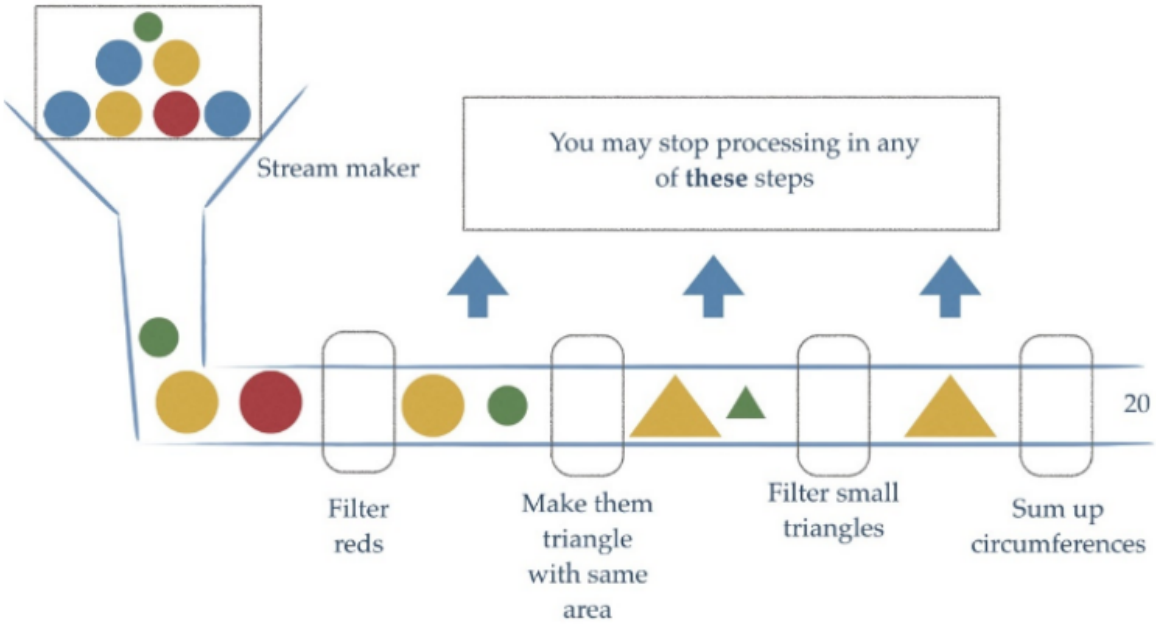
4.2.6 小结

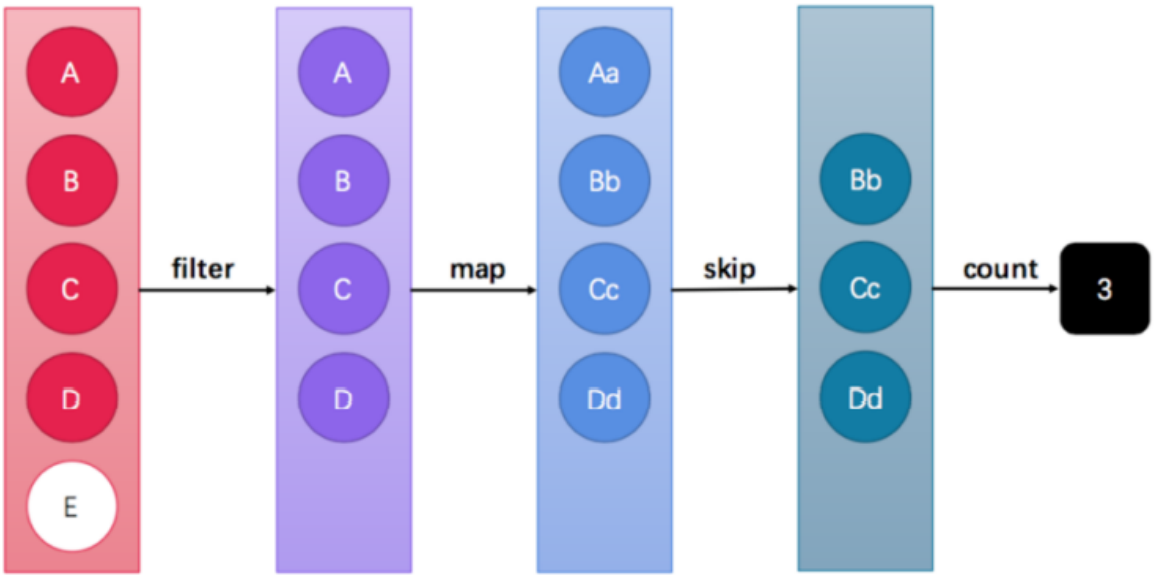
- 小结：方法引用是对Lambda表达式符合特定情况下的一种缩写方式，它使得我们的Lambda表达式更加的精简，也可以理解为lambda表达式的缩写形式，不过要注意的是方法引用只能引用已经存在的方法。

5.Stream流

5.1 Stream流简介

- Steam流式思想概述
  - Stream流式思想类似于工厂车间的“生产流水线”，Stream流不是一种数据结构，不保存数据，而是对数据进行加工处理。Stream可以看作是流水线上的一个工序。在流水线上，通过多个工序让一个原材料加工成一个商品。





- Stream API能让我们快速完成许多复杂的操作，如筛选、切片、映射、查找、去除重复，统计，匹配和归约。
- Stream流的三类方法
  - 获取Stream流
    - 创建一条流水线,并把数据放到流水线上准备进行操作

- 中间方法
  - 流水线上的操作
  - 一次操作完毕之后,还可以继续进行其他操作，返回值是一个新的Stream流
- 终结方法
  - 一个Stream流只能有一个终结方法
  - 是流水线上的最后一个操作，返回值不是Stream流

- Stream注意事项
  1. Stream只能操作一次
  2. Stream方法返回的是新的流
  3. Stream不调用终结方法，中间的操作不会执行

5.2 生成Stream流的方式

- Collection体系集合
  使用默认方法stream()生成流， default Stream stream()
- Map体系集合
  把Map转成Set集合，间接的生成流，可以先通过 keySet()、values()或者 entrySet()获取一个 Set 集合，再获取 Stream 流（双列先转单列再获取 Stream 流）。
- 数组
  通过Arrays中的静态方法stream生成流
- 同种数据类型的多个数据
  通过Stream接口的静态方法of(T... values)生成流
- 代码示例

```java
public class StreamDemo {
    public static void main(String[] args) {
        //Collection体系的集合可以使用默认方法stream()生成流
        List<String> list = new ArrayList<String>();
        Stream<String> listStream = list.stream();

        Set<String> set = new HashSet<String>();
        Stream<String> setStream = set.stream();

        //Map体系的集合间接的生成流
        Map<String,Integer> map = new HashMap<String, Integer>();
        Stream<String> keyStream = map.keySet().stream();
        Stream<Integer> valueStream = map.values().stream();
        Stream<Map.Entry<String, Integer>> entryStream = map.entrySet().stream();

        //数组可以通过Arrays中的静态方法stream生成流
        String[] strArray = {"hello","world","java"};
        Stream<String> strArrayStream = Arrays.stream(strArray);

        //同种数据类型的多个数据可以通过Stream接口的静态方法of(T... values)生成流
        Stream<String> strArrayStream2 = Stream.of("hello", "world", "java");
        Stream<Integer> intStream = Stream.of(10, 20, 30);
    }
}
```

5.3 Stream常用方法

方法名	方法作用	返回值类型	方法种类
count	统计个数	long	终结
forEach	逐一处理	void	终结
filter	过滤	Stream	函数拼接
limit	取用前几个	Stream	函数拼接
skip	跳过前几个	Stream	函数拼接
map	映射	Stream	函数拼接
concat	组合	Stream	函数拼接

5.3.1 forEach

- forEach用来遍历流中的数据
- 该方法接受一个Consumer接口，会将每一个流元素交给函数处理

```
void forEach(Consumer<? super T> action);
```

- 代码示例

```
public static void main(String[] args) {
    Stream.of("a1", "a2", "a3").forEach(System.out::println);;
}
```

5.3.2 count

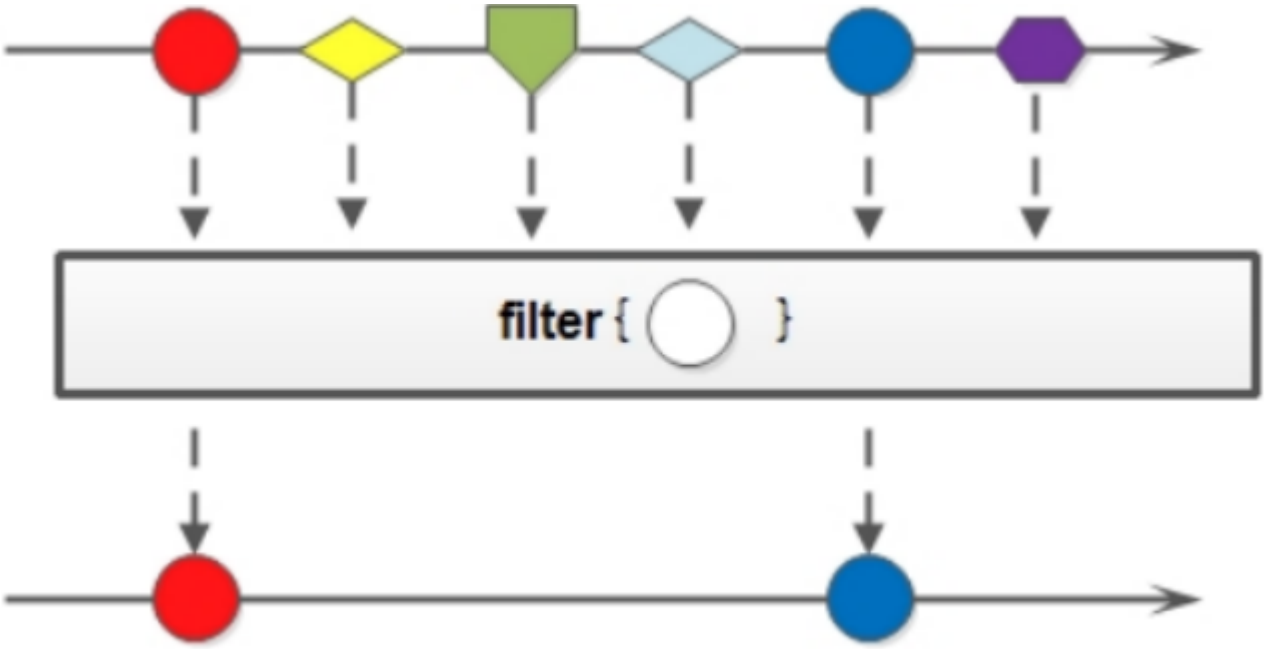
- Stream流中的count方法用来统计其中的元素个数
- 该方法返回一个long值，代表元素的个数。

```
long count();
```

- 代码示例

```
public static void main(String[] args) {
    long count = Stream.of("a1", "a2", "a3").count();
    System.out.println(count);
}
```

5.3.3 filter



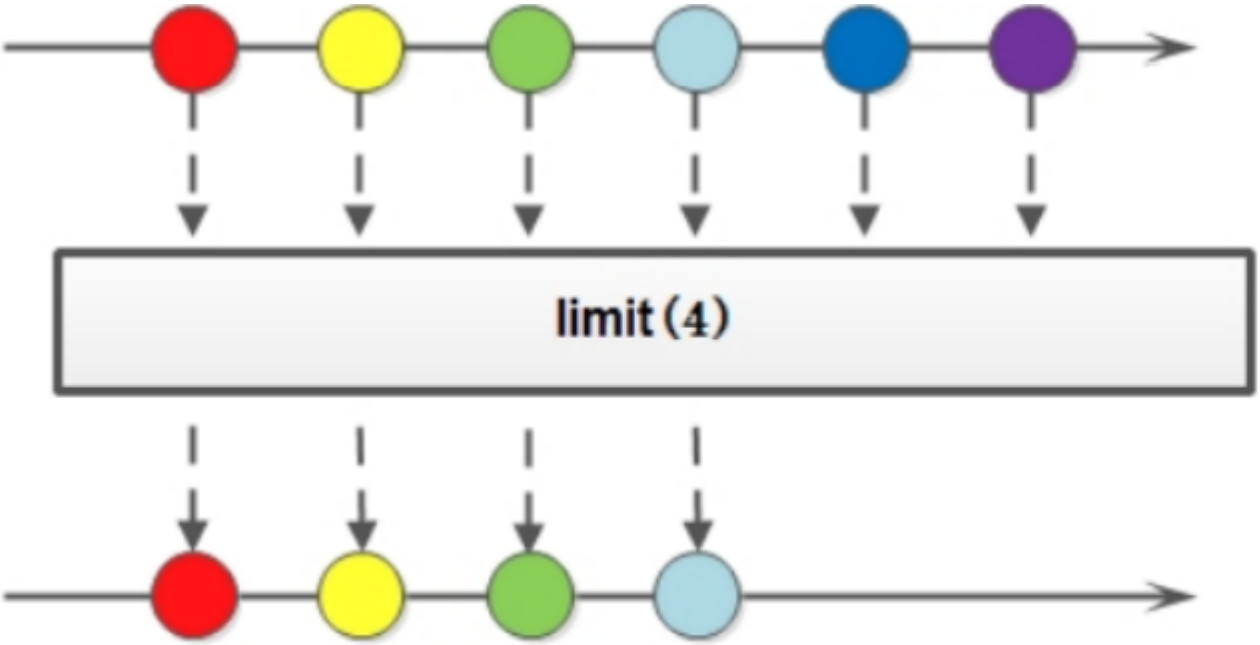
- filter方法的作用是用来过滤数据。返回符合条件的数据
- 可以通过filter方法将一个流转换成另一个子集流
- 该接口接收一个Predicate函数式接口参数作为筛选条件

```
Stream<T> filter(Predicate<? super T> predicate);
```

- 代码示例

```
public static void main(String[] args) {
    Stream.of("a1", "a2", "a3","bb","cc","aa","dd")
          .filter((s)->s.contains("a"))
          .forEach(System.out::println);
}
```

5.3.4 limit



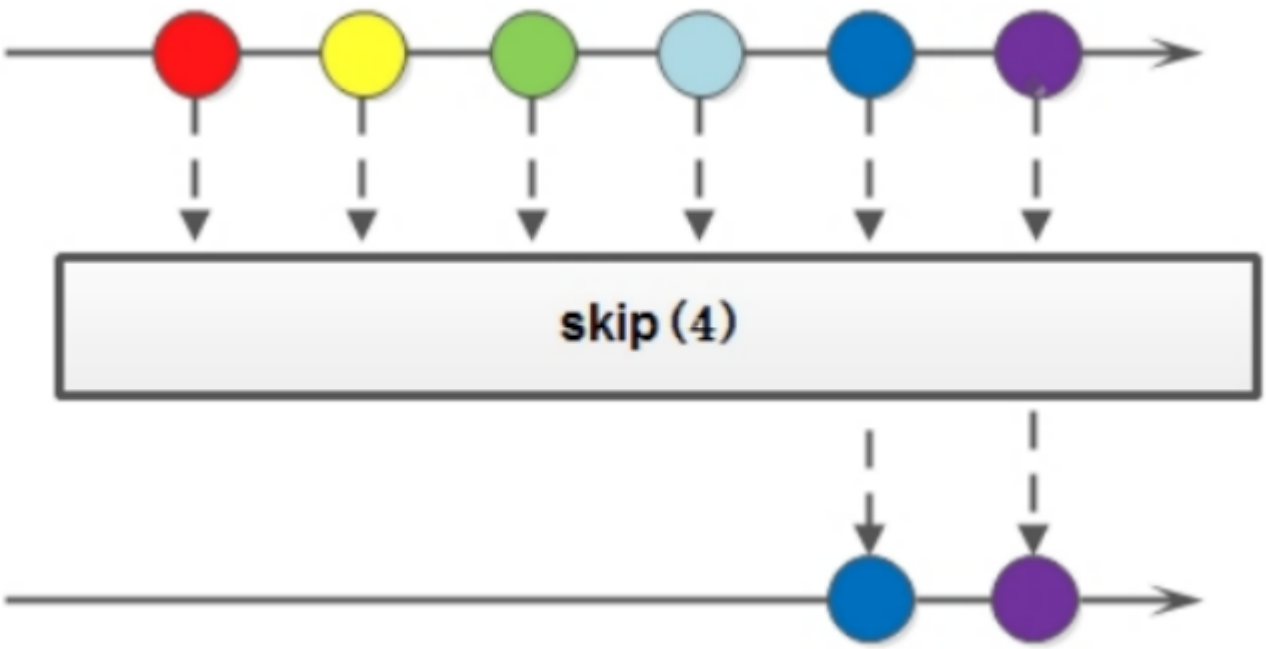
- limit方法可以对流进行截取处理，支取前n个数据
- 参数是一个long类型的数值，如果集合当前长度大于参数就进行截取，否则不操作

```
Stream<T> limit(long maxSize);
```

- 代码示例

```
public static void main(String[] args) {
    Stream.of("a1", "a2", "a3","bb","cc","aa","dd")
          .limit(3)
          .forEach(System.out::println);
}
```

5.3.5 skip

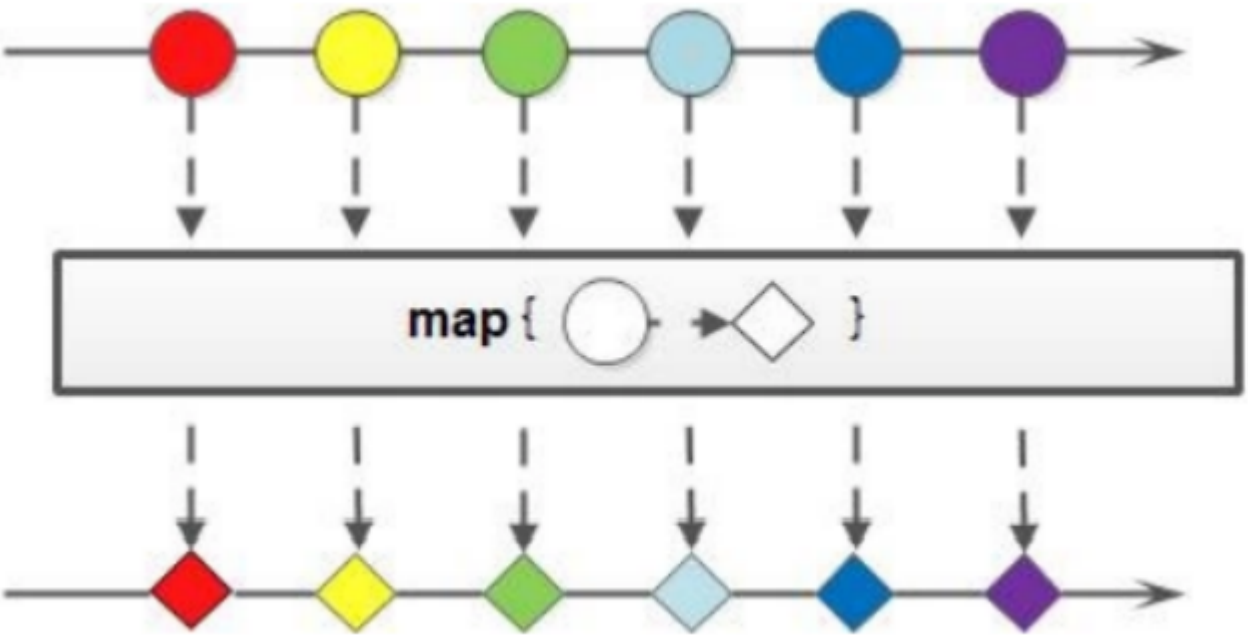


- 如果希望跳过前面几个元素，可以使用skip方法获取一个截取之后的新流

```
Stream<T> skip(long n);
```

```
public static void main(String[] args) {
    Stream.of("a1", "a2", "a3","bb","cc","aa","dd")
          .skip(3)
          .forEach(System.out::println);
}
```

5.3.6 map



- 如果我们需要将流中的元素映射到另一个流中，可以使用map方法
- 该接口需要一个Function函数式接口参数，可以将当前流中的T类型数据转换为另一种R类型的数据

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper);
```

- 代码示例

```
public static void main(String[] args) {
    Stream.of("1", "2", "3","4","5","6","7")
    //.map(msg->Integer.parseInt(msg))
          .map(Integer::parseInt)
          .forEach(System.out::println);
}
```

5.3.7 sorted

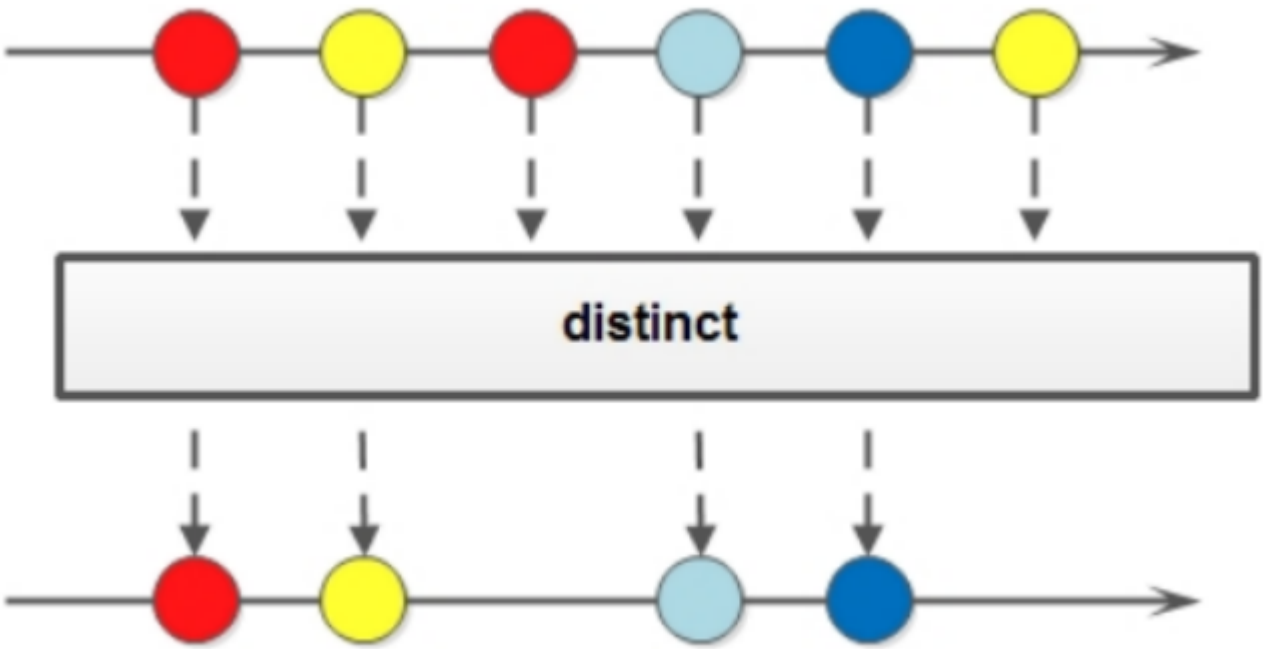
- sorted方法用于对数据排序
- 在使用的时候可以根据自然规则排序，也可以通过比较强来指定对应的排序规则

```
Stream<T> sorted();
```

- 代码示例

```
public static void main(String[] args) {
    Stream.of("1", "3", "2","4","0","9","7")
    //.map(msg->Integer.parseInt(msg))
          .map(Integer::parseInt)
    //.sorted() // 根据数据的自然顺序排序
          .sorted((o1,o2)->o2-o1) // 根据比较强指定排序规则
          .forEach(System.out::println);
}
```

5.3.8 distinct



- distinct方法去除重复元素

```
Stream<T> distinct();
```

- 代码示例

```
public static void main(String[] args) {
    Stream.of("1", "3", "3","4","0","1","7")
    //.map(msg->Integer.parseInt(msg))
          .map(Integer::parseInt)
    //.sorted() // 根据数据的自然顺序排序
          .sorted((o1,o2)->o2-o1) // 根据比较强指定排序规则
          .distinct() // 去掉重复的记录
          .forEach(System.out::println);
          System.out.println("--------");
          Stream.of(
              new Person("张三",18)
              ,new Person("李四",22)
              ,new Person("张三",18)
            ).distinct()
             .forEach(System.out::println);
}
```

- Stream流中的distinct方法对于基本数据类型是可以直接出重的，但是对于自定义类型，我们是需要重写hashCode和equals方法来移除重复元素。

5.3.9 match

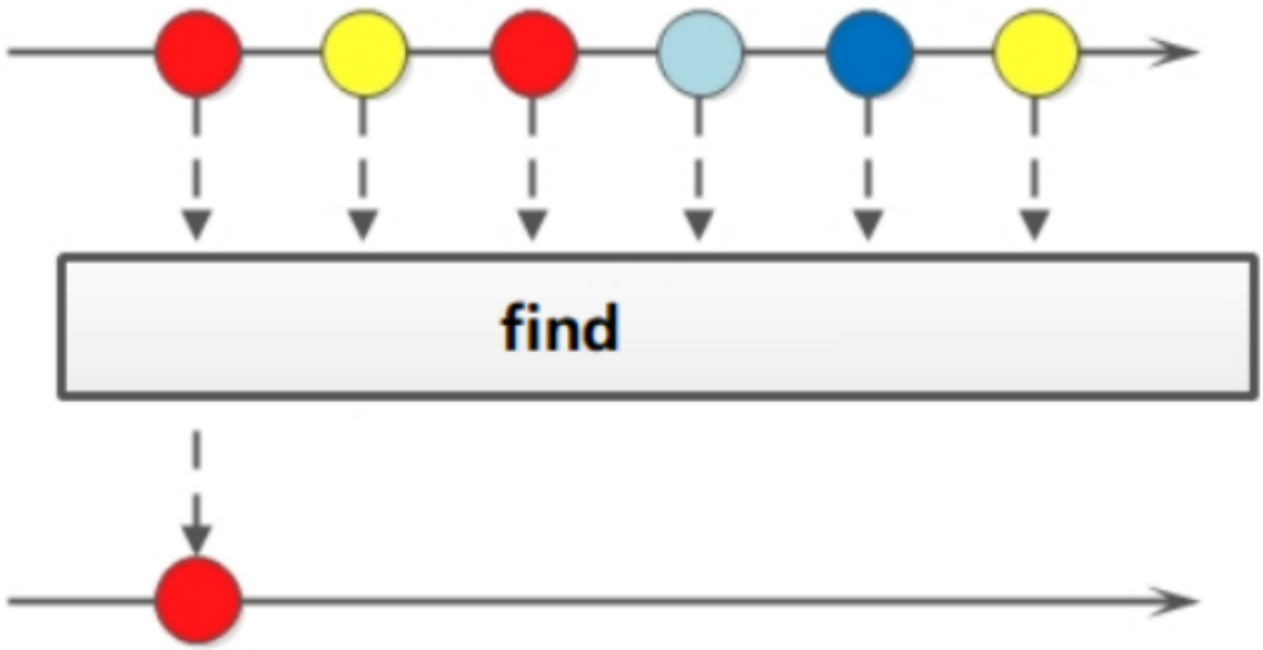
- match相关的方法用于判断数据是否匹配指定的条件

```
boolean anyMatch(Predicate<? super T> predicate); // 元素是否有任意一个满足条件
boolean allMatch(Predicate<? super T> predicate); // 元素是否都满足条件
boolean noneMatch(Predicate<? super T> predicate); // 元素是否都不满足条件
```

- 代码示例

```
public static void main(String[] args) {
    boolean b = Stream.of("1", "3", "3", "4", "5", "1", "7")
                      .map(Integer::parseInt)
                    //.allMatch(s -> s > 0)
                    //.anyMatch(s -> s >4)
                      .noneMatch(s -> s > 4);
    System.out.println(b);
}
```

5.3.10 find



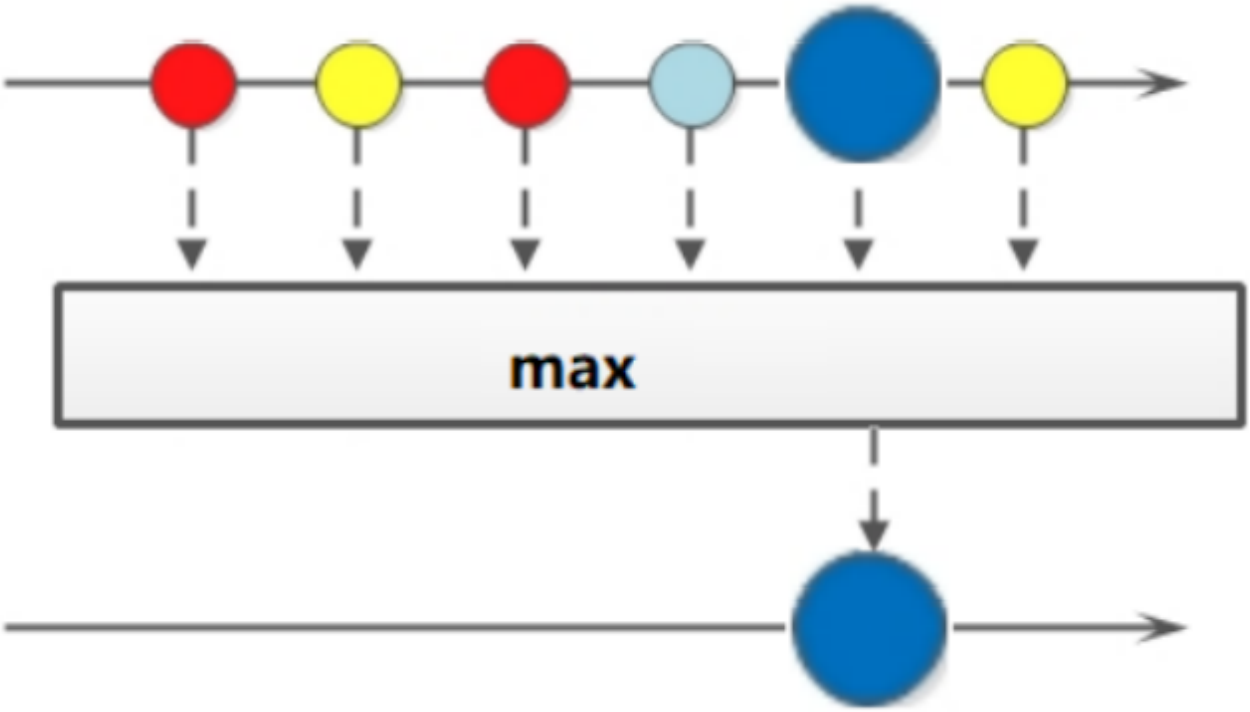
- find方法用于寻找某些符合条件的数据

```
Optional<T> findFirst();
Optional<T> findAny();
```

- 代码示例

```
public static void main(String[] args) {
    Optional<String> first = Stream.of("1", "3", "3", "4", "5", "1","7")
                                   .findFirst();
    System.out.println(first.get());
    Optional<String> any = Stream.of("1", "3", "3", "4", "5", "1","7")
                                  .findAny();
    System.out.println(any.get());
}
```

5.3.11 max和min



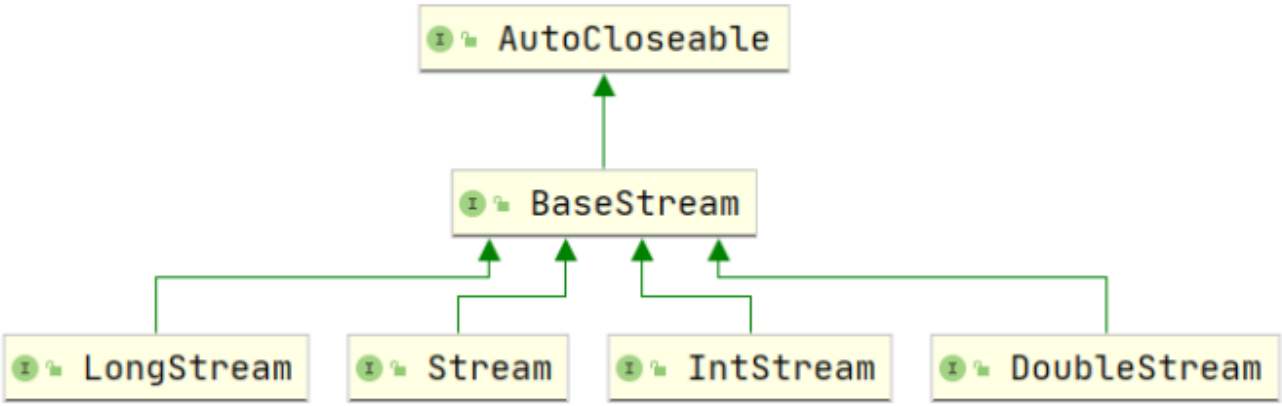
- max和min方法用于获取最大值和最小值

```
Optional<T> min(Comparator<? super T> comparator);
Optional<T> max(Comparator<? super T> comparator);
```

- 代码示例

```
public static void main(String[] args) {
    Optional<Integer> max = Stream.of("1", "3", "3", "4", "5", "1", "7")
                                .map(Integer::parseInt)
                                .max((o1,o2)->o1-o2);
    System.out.println(max.get());

    Optional<Integer> min = Stream.of("1", "3", "3", "4", "5", "1", "7")
                                .map(Integer::parseInt)
                                .min((o1,o2)->o1-o2);
    System.out.println(min.get());
}
```

5.3.12 mapToInt



- mapToInt方法用于将Stream中的Integer类型转换成int类型
- 代码示例

```
public static void main(String[] args) {
    // Integer占用的内存比int多很多，在Stream流操作中会自动装修和拆箱操作
    Integer arr[] = {1,2,3,5,6,8};
    Stream.of(arr)
          .filter(i->i>0)
          .forEach(System.out::println);
    System.out.println("---------");

    // 为了提高程序代码的效率，我们可以先将流中Integer数据转换为int数据，然后再操作
    IntStream intStream = Stream.of(arr)
                                .mapToInt(Integer::intValue);
    intStream.filter(i->i>3)
             .forEach(System.out::println);
}
```

5.3.13 concat

- concat方法用于将两个流合为一个流

```
public static <T> Stream<T> concat(Stream<? extends T> a, Stream<? extends
T> b) {
    Objects.requireNonNull(a);
    Objects.requireNonNull(b);

    @SuppressWarnings("unchecked")
    Spliterator<T> split = new Streams.ConcatSpliterator.OfRef<>((Spliterator<T>)
a.spliterator(), (Spliterator<T>)
    b.spliterator());

    Stream<T> stream = StreamSupport.stream(split, a.isParallel() ||
    b.isParallel());
    return stream.onClose(Streams.composedClose(a, b));
}
```

- 代码示例

```
public static void main(String[] args) {
    Stream<String> stream1 = Stream.of("a","b","c");
    Stream<String> stream2 = Stream.of("x", "y", "z");
    // 通过concat方法将两个流合并为一个新的流
    Stream.concat(stream1,stream2).forEach(System.out::println);
}
```

5.4 Stream流的高级操作

5.4.1 collect

- collect 是一个非常有用的终端操作，它可以将流中的元素转变成另外一个不同的对象，例如一个List，Set或Map。collect 接受入参为Collector（收集器），它由四个不同的操作组成：供应器（supplier）、累加器（accumulator）、组合器（combiner）和终止器（finisher）。
- Java 8通过Collectors类内置了各种常用的收集器

1. 对Stream流中数据进行收集

- 代码示例

```
/**
* Stream结果收集
* 收集到集合中
*/
@Test
public void test01(){
    // Stream<String> stream = Stream.of("aa", "bb", "cc");
    List<String> list = Stream.of("aa", "bb", "cc","aa")
                            .collect(Collectors.toList());
    System.out.println(list);

    // 收集到 Set集合中
    Set<String> set = Stream.of("aa", "bb", "cc", "aa")
                          .collect(Collectors.toSet());
    System.out.println(set);

    // 如果需要获取的类型为具体的实现，比如：ArrayList HashSet
    ArrayList<String> arrayList = Stream.of("aa", "bb", "cc", "aa")
            //.collect(Collectors.toCollection(() -> new ArrayList<>()));
                                    .collect(Collectors.toCollection(ArrayList::new));
    System.out.println(arrayList);

    HashSet<String> hashSet = Stream.of("aa", "bb", "cc", "aa")
                                  .collect(Collectors.toCollection(HashSet::new));
    System.out.println(hashSet);
}
```

- (拓展收集结果到数组的方法，收集结果到数组中不是collect操作)Stream中提供了toArray方法来将结果放到一个数组中，返回值类型是Object[],如果我们要指定返回的类型，那么可以使用另一个重载的toArray(IntFunction f)方法
- 收集结果到数组中
- 代码示例

```
/**
* Stream结果收集到数组中
*/
@Test
public void test02(){
    Object[] objects = Stream.of("aa", "bb", "cc", "aa")
                            .toArray(); // 返回的数组中的元素是 Object类型
    System.out.println(Arrays.toString(objects));

    // 如果我们需要指定返回的数组中的元素类型
    String[] strings = Stream.of("aa", "bb", "cc", "aa")
                            .toArray(String[]::new);
    System.out.println(Arrays.toString(strings));
}
```

2. 对Stream流中的数据做聚合计算

- 当我们使用Stream流处理数据后，可以像数据库的聚合函数一样对某个字段进行操作，比如获得最大值，最小值，求和，平均值，统计数量
- 代码示例

```java
/**
* Stream流中数据的聚合计算
*/
@Test
public void test03(){
    // 获取年龄的最大值
    Optional<Person> maxAge = Stream.of(
        new Person("张三", 18),
        new Person("李四", 22),
        new Person("张三", 13),
        new Person("王五", 15),
        new Person("张三", 19)
        ).collect(Collectors.maxBy((p1, p2) -> p1.getAge() - p2.getAge))
    System.out.println("最大年龄: " + maxAge.get());

    // 获取年龄的最小值
    Optional<Person> minAge = Stream.of(
        new Person("张三", 18),
        new Person("李四", 22),
        new Person("张三", 13),
        new Person("王五", 15),
        new Person("张三", 19)
        ).collect(Collectors.minBy((p1, p2) -> p1.getAge() - p2.getAge()));
    System.out.println("最新年龄:" + minAge.get());

    // 求所有人的年龄之和
    Integer sumAge = Stream.of(
        new Person("张三", 18),
        new Person("李四", 22),
        new Person("张三", 13),
        new Person("王五", 15),
        new Person("张三", 19)
    )
  //.collect(Collectors.summingInt(s -> s.getAge()))
    .collect(Collectors.summingInt(Person::getAge));
    System.out.println("年龄总和: " + sumAge);

    // 年龄的平均值
    Double avgAge = Stream.of(
        new Person("张三", 18),
        new Person("李四", 22),
        new Person("张三", 13),
        new Person("王五", 15),
        new Person("张三", 19)
        ).collect(Collectors.averagingInt(Person::getAge));
    System.out.println("年龄的平均值: " + avgAge);

    // 统计数量
    Long count = Stream.of(
        new Person("张三", 18),
        new Person("李四", 22),
        new Person("张三", 13),

        new Person("王五", 15),
```

```
        new Person("张三", 19)
    ).filter(p->p.getAge() > 18)
     .collect(Collectors.counting());
    System.out.println("满足条件的记录数:" + count);
}
```

- Collectors.summarizingInt()可以得到一个更全面的统计信息，摘要收集器可以返回一个特殊的内置统计对象。通过它，我们可以简单地计算出最小年龄、最大年龄、平均年龄、总和以及总数量。
- 代码示例

```
IntSummaryStatistics ageSummary =
    persons
        .stream()
        .collect(Collectors.summarizingInt(p -> p.age)); // 生成摘要统计

System.out.println(ageSummary);
// IntSummaryStatistics{count=4, sum=76, min=12, average=19.000000, max=23}
```

3. 对Stream流中数据做分组操作

- 当我们使用Stream流处理数据后，可以根据某个属性将数据分组
- 代码示例

```
/**
* 分组计算
*/
@Test
public void test04(){
    // 根据账号对数据进行分组
    Map<String, List<Person>> map1 = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).collect(Collectors.groupingBy(Person::getName));
    map1.forEach((k,v)-> System.out.println("k=" + k +"\t"+ "v=" + v));System.out.println("-----
------");

    // 根据年龄分组 如果大于等于18 成年否则未成年
    Map<String, List<Person>> map2 = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).collect(Collectors.groupingBy(p -> p.getAge() >= 18 ? "成年" : "未成
年"));
    map2.forEach((k,v)-> System.out.println("k=" + k +"\t"+ "v=" + v));
}
```

- 输出结果

```
height=166}]
k=张三 v=[Person{name='张三', age=18, height=175}, Person{name='张三', age=14,
height=165}, Person{name='张三', age=19, height=182}]
-----------
k=未成年 v=[Person{name='张三', age=14, height=165}, Person{name='李四', age=15,
height=166}]
k=成年 v=[Person{name='张三', age=18, height=175}, Person{name='李四', age=22,
height=177}, Person{name='张三', age=19, height=182}]
```

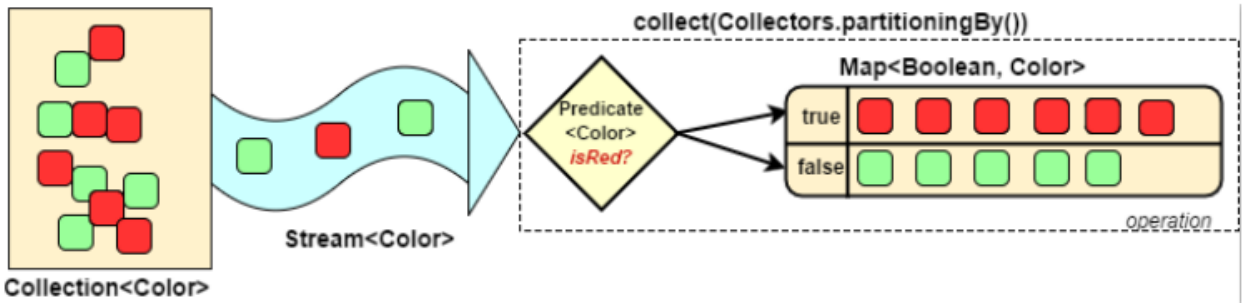
- 多级分组: 先根据name分组然后根据年龄分组
- 代码示例

```
/**
* 分组计算--多级分组
*/
@Test
public void test05(){
    // 先根据name分组，然后根据age(成年和未成年)分组
    Map<String,Map<Object,List<Person>>> map = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).collect(Collectors.groupingBy(Person::getName,Collectors.groupingBy (p->p.getAge()>=18?"成
年":"未成年")
    ));

    map.forEach((k,v)->{
        System.out.println(k);
        v.forEach((k1,v1)->{System.out.println("\t"+k1 + "=" + v1);});
    });
}
```

- 输出结果

```
李四
未成年=[Person{name='李四', age=15, height=166}]
成年=[Person{name='李四', age=22, height=177}]
张三
未成年=[Person{name='张三', age=14, height=165}]
成年=[Person{name='张三', age=18, height=175}, Person{name='张三', age=19,
height=182}]
```

4. 对Stream流中的数据做分区操作



- Collectors.partitioningBy会根据值是否为true,把集合中的数据分割为两个列表，一个true列表，一个false列表
- 代码示例

```
/**
* 分区操作
*/
@Test
public void test06(){
    Map<Boolean, List<Person>> map = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).collect(Collectors.partitioningBy(p -> p.getAge() > 18));
    map.forEach((k,v)-> System.out.println(k+"\t" + v));
}
```

- 输出结果

```
false [Person{name='张三', age=18, height=175}, Person{name='张三', age=14,
height=165}, Person{name='李四', age=15, height=166}]
true [Person{name='李四', age=22, height=177}, Person{name='张三', age=19,
height=182}]
```

5. 对Stream流中的数据做拼接

- Collectors.joining会根据指定的连接符，将所有的元素连接成一个字符。连接收集器的入参接受分隔符，以及可选的前缀以及后缀。
- 代码示例

```java
/**
* 对流中的数据做拼接操作
*/
@Test
public void test07(){
    String s1 = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).map(Person::getName)
     .collect(Collectors.joining());
     // 张三李四张三李四张三
    System.out.println(s1);

    String s2 = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).map(Person::getName)
     .collect(Collectors.joining("_"));
    // 张三_李四_张三_李四_张三
    System.out.println(s2);

    String s3 = Stream.of(
        new Person("张三", 18, 175),
        new Person("李四", 22, 177),
        new Person("张三", 14, 165),
        new Person("李四", 15, 166),
        new Person("张三", 19, 182)
    ).map(Person::getName)
     .collect(Collectors.joining("_", "###", "$$$"));
    // ###张三_李四_张三_李四_张三$$$
    System.out.println(s3);
}
```

6. 自定义收集器

- 通过Collector.of()创建一个新的收集器。同时，我们还需要传入收集器的四个组成部分：供应器、累加器、组合器和终止器。
- 代码示例

```java
//将流中的所有人转换成一个字符串，包含所有大写的名称，并以|分割
Collector<Person, StringJoiner, String> personNameCollector =
    Collector.of(
        () -> new StringJoiner(" | "),          // supplier 供应器
        (j, p) -> j.add(p.name.toUpperCase()),  // accumulator 累加器
        (j1, j2) -> j1.merge(j2),               // combiner 组合器
        StringJoiner::toString);                // finisher 终止器

String names = persons
    .stream()
    .collect(personNameCollector); // 传入自定义的收集器

System.out.println(names);  // MAX | PETER | PAMELA | DAVID
```

1. 由于Java 中的字符串是 final 类型的，我们需要借助辅助类StringJoiner，来帮我们构造字符串。
2. 最开始供应器使用分隔符构造了一个StringJointer。
3. 累加器用于将每个人的人名转大写，然后加到StringJointer中。
4. 组合器将两个StringJointer合并为一个。
5. 最终，终结器从StringJointer构造出预期的字符串。

5.4.2 flatMap

- 通过map操作, 可以将流中的对象转换为另一种类型，但是我们想要将一个对象转换为多个其他对象或者根本不做转换操作呢，这个时候，flatMap就派上用场了。
- flatMap 能够将流的每个元素, 转换为其他对象的流。因此，每个对象可以被转换为零个，一个或多个其他对象，并以流的方式返回。之后，这些流的内容会被放入flatMap返回的流中。
- flatMap会将每一个输入对象输入映射为一个新集合，然后把这些新集合连成一个大集合。
- 代码示例

```
List<Integer> list1 = new ArrayList<>();
        List<String> list2 = new ArrayList<>();
        List<String> list3 = new ArrayList<>();
        List<User> list4 = new ArrayList<>();
        List<User> list5 = new ArrayList<>();
        list1.add(1);
        list1.add(2);
        list1.add(3);
        list2.add("aa");
        list2.add("bb");
        list2.add("cc");
        list3.add("a");
        list3.add("b");
        list3.add("c");
        list4.add(new User(1,"小红a"));
        list4.add(new User(2,"小明a"));
        list5.add(new User(3,"小红B"));
        list5.add(new User(4,"小明B"));
        //1。将三个list合为一个
        final List<? extends Serializable> collect = Stream.of(list1, list2, list3)
                .flatMap(l -> l.stream())
                .collect(Collectors.toList());
        System.out.println(collect);

        //2.将两个List<Integer>合2为一
        List<String> list = Stream.of(list2, list3)
                                .flatMap(s -> s.stream())
                                .collect(Collectors.toList());
        System.out.println(list);

        //3.获取两个List<User>中得所有名字name集合
        List<String> collect1 = Stream.of(list4, list5).flatMap(s ->
s.stream().map(User::getName)).collect(Collectors.toList());
        System.out.println(collect1.toString());
```

- 输出结果

```
[1, 2, 3, aa, bb, cc, a, b, c]
[aa, bb, cc, a, b, c]
[小红a, 小明a, 小红B, 小明B]
```

5.4.3 reduce

- 规约操作可以将流的所有元素组合成一个结果
- Java 8 支持三种不同的reduce方法
- 以下操作都使用下列类

```java
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return name;
    }
}

// 构建一个 Person 集合
List<Person> persons =
    Arrays.asList(
        new Person("Max", 18),
        new Person("Peter", 23),
        new Person("Pamela", 23),
        new Person("David", 12));
```

1. reduce方法将流中的元素规约成流中的一个元素。

- 代码示例

```java
   //筛选出年龄最大的那个人
persons
    .stream()
    .reduce((p1, p2) -> p1.age > p2.age ? p1 : p2)
    .ifPresent(System.out::println);    // Pamela
```

- reduce方法接受BinaryOperator积累函数。该函数实际上是两个操作数类型相同的BiFunction。BiFunction功能和Function一样，但是它接受两个参数。示例代码中，我们比较两个人的年龄，来返回年龄较大的人。

2. reduce方法接受标识值和BinaryOperator累加器

- 代码示例

```java
//构造一个新的 Person，其中包含来自流中所有其他人的聚合名称和年龄
Person result =
    persons
        .stream()
        .reduce(new Person("", 0), (p1, p2) -> {
            p1.age += p2.age;
            p1.name += p2.name;
            return p1;
        });

System.out.format("name=%s; age=%s", result.name, result.age);
// name=MaxPeterPamelaDavid; age=76
```

3. reduce方法接受三个参数：标识值，BiFunction累加器和类型的组合器函数BinaryOperator

- 代码示例

```
//由于初始值的类型不一定为Person，我们可以使用这个归约函数来计算所有人的年龄总和
Integer ageSum = persons
    .stream()
    .reduce(0, (sum, p) -> sum += p.age, (sum1, sum2) -> sum1 + sum2);

System.out.println(ageSum);  // 76
```

5.5 Stream流执行顺序

- 中间操作的有个重要特性 —— 延迟性，没有终端操作，中间操作不会执行
- Stream 输出的结果是随着链条垂直移动的。比如说，当 Stream 开始处理第一个元素时，它实际上会在执行完 filter 操作后，再执行 forEach 操作，接着才会处理第二个元素。
- 代码示例

```
Stream.of("d2", "a2", "b1", "b3", "c")
    .filter(s -> {
        System.out.println("filter: " + s);
        return true;
    })
    .forEach(s -> System.out.println("forEach: " + s));
```

- 输出结果

```
filter:  d2
forEach: d2
filter:  a2
forEach: a2
filter:  b1
forEach: b1
filter:  b3
forEach: b3
filter:  c
forEach: c
```

- 垂直执行的原因是出于性能的考虑，这样设计可以减少对每个元素的实际操作数。
- sorted方法不是垂直执行的，sorted方法是水平执行，sorted 是一个有状态的操作，因为它需要在处理的过程中，保存状态以对集合中的元素进行排序。

总结：

Stream流的元素的执行顺序，取决于流的串并行、流的数据源、以及中间操作：

1. 数据源：如果数据源本身是无序的，那么讨论元素的执行顺序就没有意义；
2. 对于串行的流，其数据源是有序的，如果中间操作中没有排序之类的影响顺序的操作，那么在最终操作中处理元素的顺序，和数据源中元素的顺序就是一致的；如果中间操作中有排序之类的操作，那么在最终操作中处理元素的顺序，和依次执行各个中间操作之后的元素顺序，是一致的。

3. 对于并行的流，其数据源是有序的，但是其最终操作中处理元素的顺序依然是随机的；但是并行流可以通过foreachOrdered保证执行顺序和数据源中元素的顺序一致。

5.6 并行流

5.6.1 串行流

- 前面使用的Stream流都是串行，也就是在一个线程上面执行。
- 代码示例

```
/**
* 串行流
*/
@Test
public void test01(){
    Stream.of(5,6,8,3,1,6)
    .filter(s->{
        System.out.println(Thread.currentThread() + "" + s);
        return s > 3;
    }).count();
}
```

- 输出结果

```
Thread[main,5,main]5
Thread[main,5,main]6
Thread[main,5,main]8
Thread[main,5,main]3
Thread[main,5,main]1
Thread[main,5,main]6
```

5.6.2 并行流简介

- parallelStream其实就是一个并行执行的流，它通过默认的ForkJoinPool，开多线程执行流操作，可以提高多线程任务的速度。

5.6.3 并行流操作

- 获取并行流
    1. 通过List接口中的parallelStream方法来获取
    2. 通过已有的串行流转换为并行流(parallel)
- 代码示例

```java
/**
* 获取并行流的两种方式
*/
@Test
public void test02(){
    List<Integer> list = new ArrayList<>();

    // 通过List 接口 直接获取并行流
    Stream<Integer> integerStream = list.parallelStream();

    // 将已有的串行流转换为并行流
    Stream<Integer> parallel = Stream.of(1, 2, 3).parallel();
}
```

- 执行并行流操作
- 代码示例

```java
/**
* 并行流操作
*/
@Test
public void test03(){
    Stream.of(1,4,2,6,1,5,9)
    .parallel() // 将流转换为并发流，Stream处理的时候就会通过多线程处理
    .filter(s->{
        System.out.println(Thread.currentThread() + " s=" +s);
        return s > 2;
    })
    .count();
}
```

- 输出结果

```
Thread[main,5,main] s=1
Thread[ForkJoinPool.commonPool-worker-2,5,main] s=9
Thread[ForkJoinPool.commonPool-worker-6,5,main] s=6
Thread[ForkJoinPool.commonPool-worker-13,5,main] s=2
Thread[ForkJoinPool.commonPool-worker-9,5,main] s=4
Thread[ForkJoinPool.commonPool-worker-4,5,main] s=5
Thread[ForkJoinPool.commonPool-worker-11,5,main] s=1
```

5.6.4 并行流线程安全问题

- 在多线程的处理下，肯定会出现数据安全问题。
- 代码示例

```java
@Test
public void test01(){
    List<Integer> list = new ArrayList<>();
    for (int i = 0; i < 1000; i++) {
        list.add(i);
    }
    System.out.println(list.size());
    List<Integer> listNew = new ArrayList<>();
    // 使用并行流来向集合中添加数据
    list.parallelStream()
    //.forEach(s->listNew.add(s));
        .forEach(listNew::add);
    System.out.println(listNew.size());
}
```

- 输出结果

```
839
```

或者抛异常

```
java.lang.ArrayIndexOutOfBoundsException
at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
at
sun.reflect.NativeConstructorAccessorImpl.newInstance(NativeConstructorAccessorImpl.java:62)
at
sun.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstructorA
ccessorImpl.java:45)
at java.lang.reflect.Constructor.newInstance(Constructor.java:423)
at
java.util.concurrent.ForkJoinTask.getThrowableException(ForkJoinTask.java:598)
....
Caused by: java.lang.ArrayIndexOutOfBoundsException: 366
at java.util.ArrayList.add(ArrayList.java:463)
```

- 解决方法

1. 加同步锁
2. 使用线程安全的容器
3. 通过Stream中的toArray/collect操作

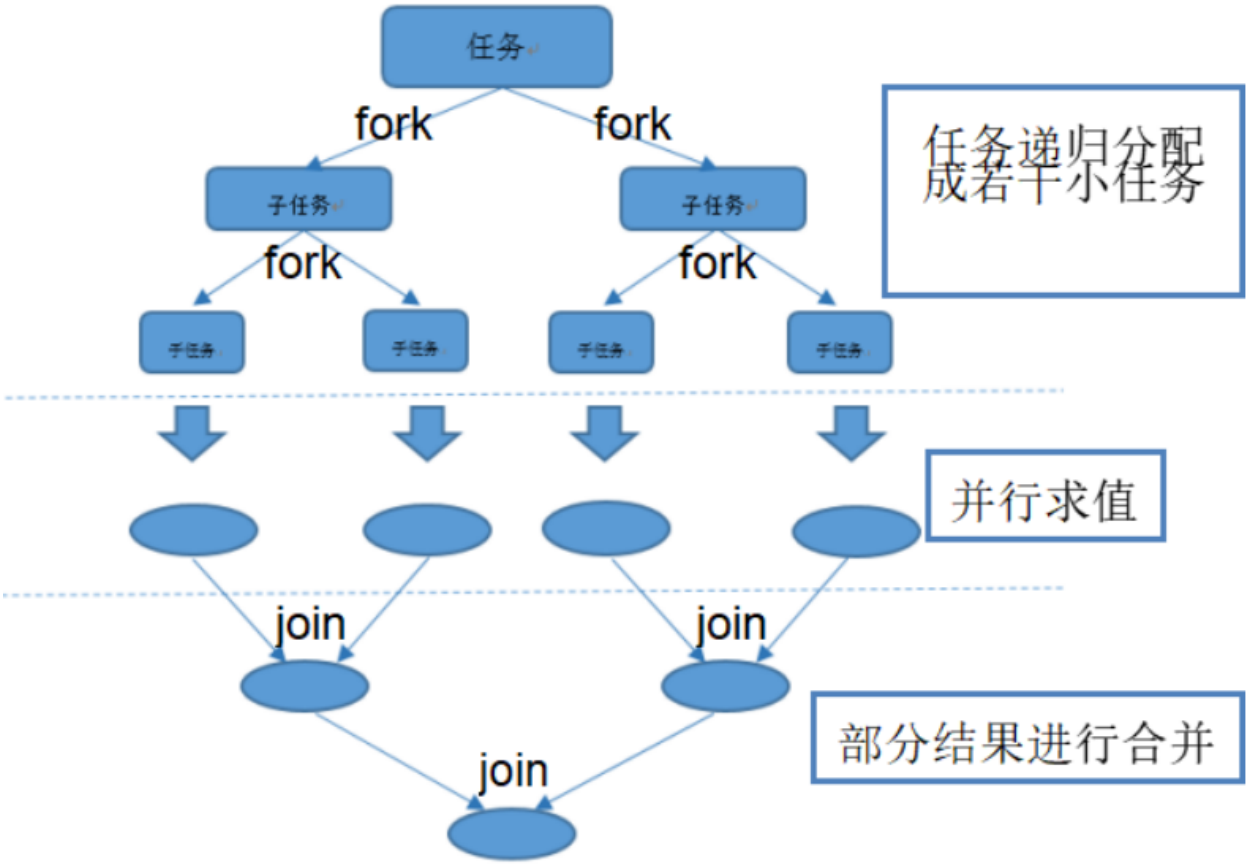
- 代码示例

```java
/**
* 加同步锁
*/
@Test
public void test02(){
    List<Integer> listNew = new ArrayList<>();
    Object obj = new Object();
    IntStream.rangeClosed(1,1000)
             .parallel()
             .forEach(i->{
                 synchronized (obj){
                     listNew.add(i);
                 }
             });
    System.out.println(listNew.size());
}

/**
* 使用线程安全的容器
*/
@Test
public void test03(){
    Vector v = new Vector();
    Object obj = new Object();
    IntStream.rangeClosed(1,1000)
             .parallel()
             .forEach(i->{
                 synchronized (obj){
                     v.add(i);
                 }
             });
    System.out.println(v.size());
}
/**
* 将线程不安全的容器转换为线程安全的容器
*/
@Test
public void test04(){
    List<Integer> listNew = new ArrayList<>();
    // 将线程不安全的容器包装为线程安全的容器
    List<Integer> synchronizedList = Collections.synchronizedList(listNew);
    Object obj = new Object();
    IntStream.rangeClosed(1,1000)
             .parallel()
             .forEach(i->{
                 synchronizedList.add(i);
             });
    System.out.println(synchronizedList.size());
}
/**
* 我们还可以通过Stream中的 toArray方法或者 collect方法来操作
* 就是满足线程安全的要求

*/
```

```java
@Test
public void test05(){
    List<Integer> listNew = new ArrayList<>();
    Object obj = new Object();
    List<Integer> list = IntStream.rangeClosed(1, 1000)
                                .parallel()
                                .boxed()
                                .collect(Collectors.toList());
    System.out.println(list.size());
}
```

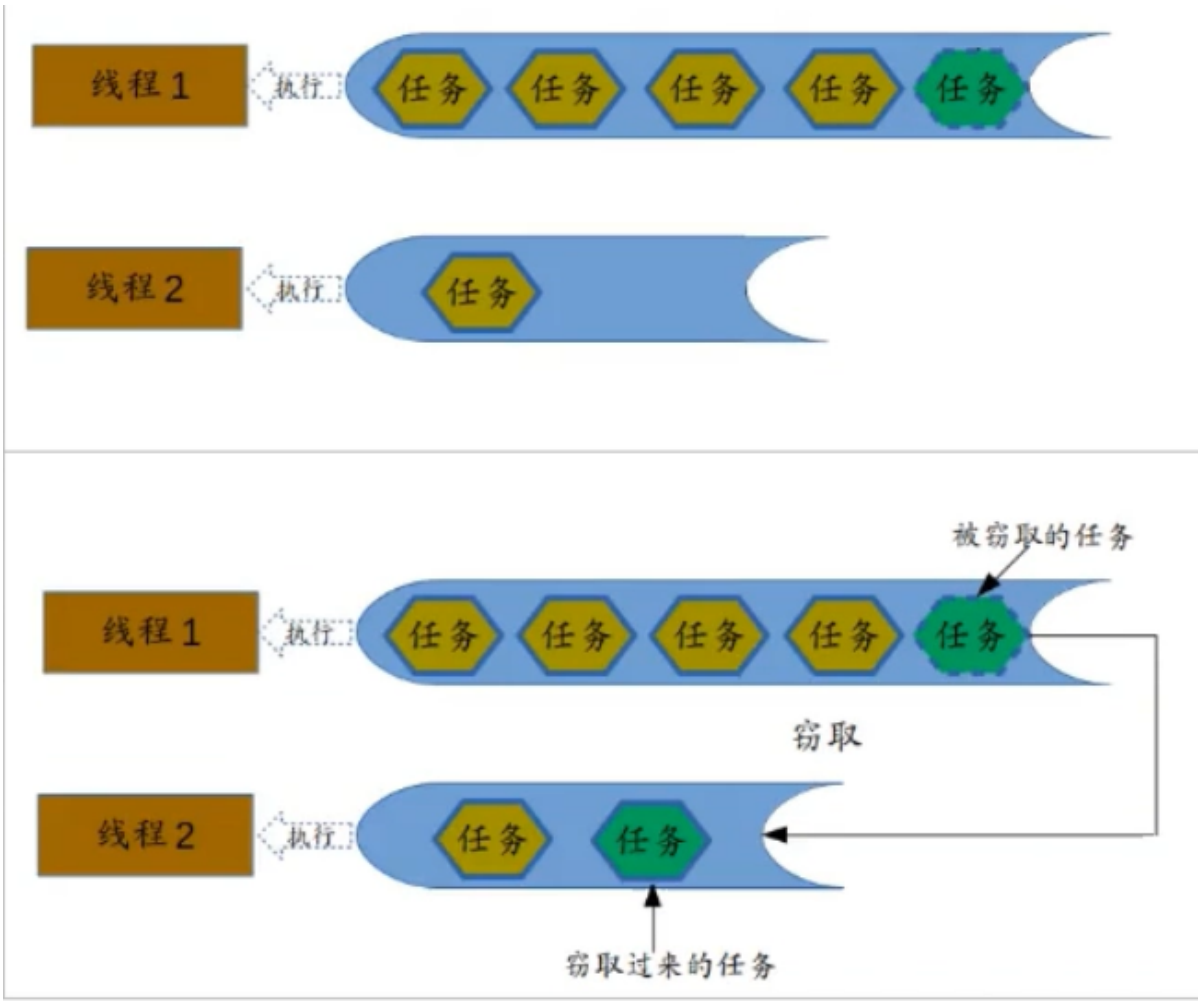
5.6.5 并行流原理

- 对于并行流，其在底层实现中，是沿用了Java7提供的fork/join分解合并框架进行实现。fork根据cpu核数进行数 据分块，join对各个fork进行合并。



- Fork/Join框架：Java7提供的一个用于并行执行任务的框架， 是一个把大任务分割成若干个小任务，最终汇总每个小任务结果后得到大任务结果的框架。
- Fork/Join框架主要包括三个部分
    1. 线程池：ForkJoinPool
    2. 任务对象：ForkJoinTask
    3. 执行任务的线程 ForkJoinWorkerThread
- Fork/Join框架原理-分治法
    - 分治法：把一个规模大的问题划分为规模较小的子问题，然后分而治之，最后合并子问题的解得到原问题的解。
    - 步骤：

    1. 分割原问题;
    2. 求解子问题;
    3. 合并子问题的解为原问题的解。
  - 在分治法中，子问题一般是相互独立的，因此，经常通过递归调用算法来求解子问题。
- Fork/Join框架原理-工作窃取算法
  - 工作窃取（work-stealing）算法是指某个线程从其他队列里窃取任务来执行。工作窃取的运行流程图如下：



  - 工作窃取算法的作用：假如我们需要做一个比较大的任务，我们可以把这个任务分割为若干互不依赖的子任务，为了减少线程间的竞争，于是把这些子任务分别放到不同的队列里，并为每个队列创建一个单独的线程来执行队列里的任务，线程和队列一一对应，比如A线程负责处理A队列里的任务。但是有的线程会先把自己队列里的任务干完，而其他线程对应的队列里还有任务等待处理。干完活的线程与其等着，不如去帮其他线程干活，于是它就去其他线程的队列里窃取一个任务来执行。而在这时它们会访问同一个队列，所以为了减少窃取任务线程和被窃取任务线程之间的竞争，通常会使用双端队列，被窃取任务线程永远从双端队列的头部拿任务执行，而窃取任务的线程永远从双端队列的尾部拿任务执行。
  - 工作窃取算法优缺点：工作窃取算法的优点是充分利用线程进行并行计算，并减少了线程间的竞争，其缺点是在某些情况下还是存在竞争，比如双端队列里只有一个任务时。并且消耗了更多的系统资源，比如创建多个线程和多个双端队列。

6.Optional类

6.1 Optional类简介

- Optional类主要是解决空指针的问题
- Optional是一个没有子类的工具类，Optional是一个可以为null的容器对象，它的主要作用就是为了避免Null检查，防止NullpointerException。

6.2 Optional类的基本使用

- Optional对象的创建方式

```
/**
* Optional对象的创建方式
*/
@Test
public void test02(){
    // 第一种方式 通过of方法 of方法是不支持null的
    Optional<String> op1 = Optional.of("zhangsan");
    //Optional<Object> op2 = Optional.of(null);
    // 第二种方式通过 ofNullable方法 支持null
    Optional<String> op3 = Optional.ofNullable("lisi");
    Optional<Object> op4 = Optional.ofNullable(null);
    // 第三种方式 通过empty方法直接创建一个空的Optional对象
    Optional<Object> op5 = Optional.empty();
}
```

- Optional的常用方法

```java
/**
   * Optional中的常用方法介绍
   * get(): 如果Optional有值则返回，否则抛出NoSuchElementException异常
   * get()通常和isPresent方法一块使用
   * isPresent():判断是否包含值，包含值返回true，不包含值返回false
   * orElse(T t):如果调用对象包含值，就返回该值，否则返回t
   * orElseGet(Supplier s):如果调用对象包含值，就返回该值，否则返回 Lambda表达式的返回值
*/
@Test
public void test03(){
    Optional<String> op1 = Optional.of("zhangsan");
    Optional<String> op2 = Optional.empty();
    // 获取Optional中的值
    if(op1.isPresent()){
        String s1 = op1.get();
        System.out.println("用户名称:" +s1);
    }
    if(op2.isPresent()){
        System.out.println(op2.get());
    }else{
        System.out.println("op2是一个空Optional对象");
    }
    String s3 = op1.orElse("李四");
    System.out.println(s3);
    String s4 = op2.orElse("王五");
    System.out.println(s4);
    String s5 = op2.orElseGet(()->{
        return "Hello";
    });
    System.out.println(s5);
}
```

```java
@Test
public void test04(){
    Optional<String> op1 = Optional.of("zhangsan");
    Optional<String> op2 = Optional.empty();
    // 如果存在值 就做什么
    op1.ifPresent(s-> System.out.println("有值:" +s));
    op1.ifPresent(System.out::println);
}

/**
* 自定义一个方法，将Person对象中的 name 转换为大写 并返回
*/
@Test
public void test05(){
    Person p = new Person("zhangsan",18);
    Optional<Person> op = Optional.of(p);
    String name = getNameForOptional(op);
    System.out.println("name="+name);
}

/**
* 根据Person对象 将name转换为大写并返回
* 通过Optional方式实现
* @param op
* @return
*/
public String getNameForOptional(Optional<Person> op){
    if(op.isPresent()){
        String msg = //op.map(p -> p.getName())
        op.map(Person::getName)
        //.map(p -> p.toUpperCase())
          .map(String::toUpperCase)
          .orElse("空值");
        return msg;
    }
    return null;
}

/**
* 根据Person对象 将name转换为大写并返回
* @param person
* @return
*/
public String getName(Person person){
    if(person != null){
        String name = person.getName();
        if(name != null){
            return name.toUpperCase();
        }else{
            return null;
        }
    }else{

        return null;
```

```
    }
}
```

7.新时间类API

7.1 旧版日期时间的问题

1. 设计不合理，在java.util和java.sql的包中都有日期类，java.util.Date同时包含日期和时间的，而java.sql.Date仅仅包含日期，此外用于格式化和解析的类在java.text包下。
2. 非线程安全，java.util.Date是非线程安全的，所有的日期类都是可变的，这是java日期类最大的问题之一。
3. 时区处理麻烦，日期类并不提供国际化，没有时区支持。

7.2 新版日期时间API介绍

- JDK 8中增加了一套全新的日期时间API，这套API设计合理，是线程安全的。新的日期及时间API位于java.time包中。
- 新版日期时间常用API
    - LocalDate ：表示日期，包含年月日，格式为 2019-10-16
    - LocalTime ：表示时间，包含时分秒，格式为 16:38:54.158549300
    - LocalDateTime ：表示日期时间，包含年月日，时分秒，格式2018-09-06T15:33:56.750
    - DateTimeFormatter ：日期时间格式化类。
    - Instant：时间戳，表示一个特定的时间瞬间。
    - Duration：用于计算2个时间(LocalTime，时分秒)的距离
    - Period：用于计算2个日期(LocalDate，年月日)的距离
    - ZonedDateTime ：包含时区的时间
- Java中使用的历法是ISO 8601日历系统，它是世界民用历法，也就是我们所说的公历。平年有365天，闰年是366天。
- 此外Java 8还提供了4套其他历法，分别是：

  ThaiBuddhistDate：泰国佛教历

  MinguoDate：中华民国历

  JapaneseDate：日本历

  HijrahDate：伊斯兰历

7.3 日期时间API的常见操作

- LocalDate，LocalTime以及LocalDateTime的操作。
- 代码示例

```
/**
* JDK8 日期时间操作 LocalDate
*/
@Test
public void test01(){
    // 1.创建指定的日期
    LocalDate date1 = LocalDate.of(2021, 05, 06);
    System.out.println("date1 = "+date1);
    // 2.得到当前的日期
    LocalDate now = LocalDate.now();
    System.out.println("now = "+now);
    // 3.根据LocalDate对象获取对应的日期信息
    System.out.println("年：" + now.getYear());
    System.out.println("月：" + now.getMonth().getValue());
    System.out.println("日：" + now.getDayOfMonth());
    System.out.println("星期：" + now.getDayOfWeek().getValue());
}

/**
* 时间操作 LocalTime
*/
@Test
public void test02(){
    // 1.得到指定的时间
    LocalTime time = LocalTime.of(5,26,33,23145);
    System.out.println(time);
    // 2.获取当前的时间
    LocalTime now = LocalTime.now();
    System.out.println(now);
    // 3.获取时间信息
    System.out.println(now.getHour());
    System.out.println(now.getMinute());
    System.out.println(now.getSecond());
    System.out.println(now.getNano());
}
/**
* 日期时间类型 LocalDateTime
*/
@Test
public void test03(){
    // 获取指定的日期时间
    LocalDateTime dateTime =
    LocalDateTime.of(2020,06,01,12,12,33,213);
    System.out.println(dateTime);
    // 获取当前的日期时间
    LocalDateTime now = LocalDateTime.now();
    System.out.println(now);
    // 获取日期时间信息
    System.out.println(now.getYear());
    System.out.println(now.getMonth().getValue());
    System.out.println(now.getDayOfMonth());
    System.out.println(now.getDayOfWeek().getValue());

    System.out.println(now.getHour());
```

```
    System.out.println(now.getMinute());
    System.out.println(now.getSecond());
    System.out.println(now.getNano());
}
```

7.4 日期时间的修改和比较

- 代码示例

```
/**
* 日期时间的修改
*/
@Test
public void test01(){
    LocalDateTime now = LocalDateTime.now();
    System.out.println("now = "+now);
    // 修改日期时间 对日期时间的修改，对已存在的LocalDate对象，创建了它模板
    // 并不会修改原来的信息
    LocalDateTime localDateTime = now.withYear(1998);
    System.out.println("now :"+now);
    System.out.println("修改后的：" + localDateTime);
    System.out.println("月份：" + now.withMonth(10));
    System.out.println("天：" + now.withDayOfMonth(6));
    System.out.println("小时：" + now.withHour(8));
    System.out.println("分钟:" + now.withMinute(15));
    // 在当前日期时间的基础上 加上或者减去指定的时间
    System.out.println("两天后:" + now.plusDays(2));
    System.out.println("10年后:"+now.plusYears(10));
    System.out.println("6个月后 = " + now.plusMonths(6));
    System.out.println("10年前 = " + now.minusYears(10));
    System.out.println("半年前 = " + now.minusMonths(6));
    System.out.println("一周前 = " + now.minusDays(7));
}

/**
* 日期时间的比较
*/
@Test
public void test02(){
    LocalDate now = LocalDate.now();
    LocalDate date = LocalDate.of(2020, 1, 3);
    // 在JDK8中要实现 日期的比较 isAfter isBefore isEqual 通过这几个方法来直接比较
    System.out.println(now.isAfter(date)); // true
    System.out.println(now.isBefore(date)); // false
    System.out.println(now.isEqual(date)); // false
}
```

注意：在进行日期时间修改的时候，原来的LocalDate对象是不会被修改，每次操作都是返回了一个新的LocalDate对象，所以在多线程场景下是数据安全的。

7.5 格式化和解析操作

- 在JDK8中我们可以通过 java.time.format.DateTimeFormatter 类可以进行日期的解析和格式化操作
- 代码示例

```
/**
* 日期格式化
*/
@Test
public void test01(){
    LocalDateTime now = LocalDateTime.now();
    // 指定格式 使用系统默认的格式 2021-05-27T16:16:38.139
    DateTimeFormatter isoLocalDateTime = DateTimeFormatter.ISO_LOCAL_DATE_TIME;
    // 将日期时间转换为字符串
    String format = now.format(isoLocalDateTime);
    System.out.println("format = " + format);
    // 通过 ofPattern 方法来指定特定的格式
    DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPatter("yyyy-MM-dd HH:mm:ss");
    String format1 = now.format(dateTimeFormatter);
    // 2021-05-27 16:16:38
    System.out.println("format1 = " + format1);
    // 将字符串解析为一个 日期时间类型
    LocalDateTime parse = LocalDateTime.parse("1997-05-06 22:45:16",dateTimeFormatter);
    // parse = 1997-05-06T22:45:16
    System.out.println("parse = " + parse);
}
```

7.6 Instant类

- 在JDK8中给我们新增一个Instant类(时间戳/时间线)，内部保存了从1970年1月1日 00:00:00以来的秒和纳秒
- 代码示例

```
/**
* Instant 时间戳
* 可以用来统计时间消耗
*/
@Test
public void test01() throws Exception{
    Instant now = Instant.now();
    System.out.println("now = " + now);

    // 获取从1970年一月一日 00:00:00 到现在的 纳秒
    System.out.println(now.getNano());
    Thread.sleep(5);
    Instant now1 = Instant.now();
    System.out.println("耗时: " + (now1.getNano() - now.getNano()));
}
```

7.7 计算日期时间差

- JDK8中提供了两个工具类Duration/Period：计算日期时间差
- 两个类都能获取两个时间对象的时间间隔，并计算时间间隔
- Period类是获得两时间对象间隔的年月日信息

- Duration类是获取两个时间对象间隔的时分秒信息，所以时间对象必须有时分秒信息

7.7.1 Period类和Duration类常用方法

- Period类

方法名	说明
public static Period between(开始时间,结束时间)	计算两个“时间"的间隔
public int getYears()	获得这段时间的年数
public int getMonths()	获得此期间的总月数
public int getDays()	获得此期间的天数
public long toTotalMonths()	获取此期间的总月数

- 代码示例

```
public class JDK8DateDemo9 {
    public static void main(String[] args) {
        //public static Period between(开始时间,结束时间)  计算两个"时间"的间隔

        LocalDate localDate1 = LocalDate.of(2020, 1, 1);
        LocalDate localDate2 = LocalDate.of(2048, 12, 12);
        Period period = Period.between(localDate1, localDate2);
        System.out.println(period);//P28Y11M11D

        //public int getYears()         获得这段时间的年数
        System.out.println(period.getYears());//28
        //public int getMonths()        获得此期间的月数
        System.out.println(period.getMonths());//11
        //public int getDays()          获得此期间的天数
        System.out.println(period.getDays());//11

        //public long toTotalMonths()   获取此期间的总月数
        System.out.println(period.toTotalMonths());//347

    }
}
```

- Duration类

方法名	说明
public static Durationbetween(开始时间,结束时间)	计算两个“时间"的间隔
public long toSeconds()	获得此时间间隔的秒
public int toMillis()	获得此时间间隔的毫秒
public int toNanos()	获得此时间间隔的纳秒

- 代码示例

```java
public class JDK8DateDemo10 {
    public static void main(String[] args) {
        //public static Duration between(开始时间,结束时间)  计算两个“时间"的间隔

        LocalDateTime localDateTime1 = LocalDateTime.of(2020, 1, 1, 13, 14, 15);
        LocalDateTime localDateTime2 = LocalDateTime.of(2020, 1, 2, 11, 12, 13);
        Duration duration = Duration.between(localDateTime1, localDateTime2);
        System.out.println(duration);//PT21H57M58S
        //public long toSeconds()          获得此时间间隔的秒
        System.out.println(duration.toSeconds());//79078
        //public int toMillis()            获得此时间间隔的毫秒
        System.out.println(duration.toMillis());//79078000
        //public int toNanos()             获得此时间间隔的纳秒
        System.out.println(duration.toNanos());//79078000000000
    }
}
```

PS:Duration.between（开始时间，结束时间）这个方法求时间间隔时，必须给带有 时 分 秒 的参数（ 如 LocalDateTime)如果参数不含秒钟时间，则会报错UnsupportedTemporalTypeException异常

7.8 时间校正器

- 有时候我们可以需要如下调整：将日期调整到"下个月的第一天"等操作。这时我们通过时间校正器效果可能会更好。
- TemporalAdjuster:时间校正器
- TemporalAdjusters:通过该类静态方法提供了大量的常用TemporalAdjuster的实现。
- 代码示例

```java
/**
* 时间校正器
*/
@Test
public void test02(){
    LocalDateTime now = LocalDateTime.now();
    // 将当前的日期调整到下个月的一号
    TemporalAdjuster adJuster = (temporal)->{
        LocalDateTime dateTime = (LocalDateTime) temporal;
        LocalDateTime nextMonth = dateTime.plusMonths(1).withDayOfMonth(1);
        System.out.println("nextMonth = " + nextMonth);
        return nextMonth;
    };
    // 我们可以通过TemporalAdjusters 来实现
    // LocalDateTime nextMonth = now.with(adJuster);
    LocalDateTime nextMonth = now.with(TemporalAdjusters.firstDayOfNextMonth());
    System.out.println("nextMonth = " + nextMonth);
}
```

7.9 日期时间的时区

- Java8 中加入了对时区的支持，LocalDate、LocalTime、LocalDateTime是不带时区的，带时区的日期时间类分别为：ZonedDate、ZonedTime、ZonedDateTime。其中每个时区都对应着 ID，ID的格式为 “区域/城市”

。例如：Asia/Shanghai 等。ZoneId：该类中包含了所有的时区信息

- 代码示例

```
/**
* 时区操作
*/
@Test
public void test01(){
    // 1.获取所有的时区id
    // ZoneId.getAvailableZoneIds().forEach(System.out::println);
    // 获取当前时间 中国使用的 东八区的时区，比标准时间早8个小时

    LocalDateTime now = LocalDateTime.now();
    System.out.println("now = " + now); // 2021-05-27T17:17:06.951
    // 获取标准时间
    ZonedDateTime bz = ZonedDateTime.now(Clock.systemUTC());
    System.out.println("bz = " + bz); // 2021-05-27T09:17:06.952Z

    // 使用计算机默认的时区，创建日期时间
    ZonedDateTime now1 = ZonedDateTime.now();
    System.out.println("now1 = " + now1);  //2021-05-27T17:17:06.952+08:00[Asia/Shanghai]

    // 使用指定的时区创建日期时间
    ZonedDateTime now2 = ZonedDateTime.now(ZoneId.of("America/Marigot"));
    System.out.println("now2 = " + now2);
}
```

7.10 JDK新的日期和时间API的优势

1. 新版日期时间API中，日期和时间对象是不可变，操作日期不会影响原来的值，而是生成一个新的实例
2. 提供不同的两种方式，有效的区分了人和机器的操作
3. TemporalAdjuster可以更精确的操作日期，还可以自定义日期调整期
4. 线程安全

8.注解新特性

8.1 重复注解

- 自从Java 5中引入注解以来，注解开始变得非常流行，并在各个框架和项目中被广泛使用。不过注解有一个很大的限制是：在同一个地方不能多次使用同一个注解。JDK 8引入了重复注解的概念，允许在同一个地方多次使用同一个注解。在JDK 8中使用@Repeatable注解定义重复注解。

1. 定义一个重复注解的容器

```
@Retention(RetentionPolicy.RUNTIME)
public @interface MyAnnotations {
    MyAnnotation[] value();
}
```

2. 定义一个可以重复的注解

```
@Repeatable(MyAnnotations.class)
@Retention(RetentionPolicy.RUNTIME)
public @interface MyAnnotation {
    String value();
}
```

3. 配置多个重复的注解

```
@MyAnnotation("test1")
@MyAnnotation("test2")
@MyAnnotation("test3")
public class AnnoTest01 {
    @MyAnnotation("fun1")
    @MyAnnotation("fun2")
    public void test01(){

    }
}
```

4. 解析得到指定的注解

```
/**
* 解析重复注解
* @param args
*/
public static void main(String[] args) throws NoSuchMethodException {
    // 获取类中标注的重复注解
    MyAnnotation[] annotationsByType =
    AnnoTest01.class.getAnnotationsByType(MyAnnotation.class);
    for (MyAnnotation myAnnotation : annotationsByType) {
        System.out.println(myAnnotation.value());
    }

    // 获取方法上标注的重复注解
    MyAnnotation[] test01s = AnnoTest01.class.getMethod("test01")
    .getAnnotationsByType(MyAnnotation.class);
    for (MyAnnotation test01 : test01s) {
        System.out.println(test01.value());
    }
}
```

8.2 类型注解

- JDK 8为@Target元注解新增了两种类型：TYPE_PARAMETER ， TYPE_USE。
  - TYPE_PARAMETER ：表示该注解能写在类型参数的声明语句中。
  - TYPE_USE ：表示注解可以再任何用到类型的地方使用。
- TYPE_PARAMETER代码示例

1. 定义注解

```
@Target(ElementType.TYPE_PARAMETER)
public @interface TypeParam {
}
```

2. 注解使用

```
public class TypeDemo01 <@TypeParam T> {
    public <@TypeParam K extends Object> K test01(){
        return null;
    }
}
```

- TYPE_USE代码示例

1. 定义注解

```
@Target(ElementType.TYPE_USE)
public @interface NotNull {
}
```

2. 注解使用

```
public class TypeUseDemo01 {
    public @NotNull Integer age = 10;
    public Integer sum(@NotNull Integer a,@NotNull Integer b){
        return a + b;
    }
}
```